

Universität Trier

Hybrid Knowledge Graph Construction from Structured Metadata and Biography Text

Master's Thesis

for obtaining the academic degree
Master of Science (M.Sc.)

University of Trier
FB IV - Computer Science

Reviewer:	Prof. Dr.-Ing. Ralf Schenkel
	JProf. Dr. Jacqueline Staub
Supervisor:	Prof. Dr.-Ing. Ralf Schenkel

submitted on 17.06.2026 by:

Carishmarahamathi Ahamed Khan
s4caaham@uni-trier.de
Matr.-Nr. 1624118

Abstract

As academic publications and researcher profiles grow, there is an increasing need for automated methods to extract and organize biographical information about researchers at scale. This thesis presents a schema-driven, prompt-based large language model pipeline for extracting structured career information from unstructured academic biography texts without requiring task-specific model fine-tuning. The pipeline serves as a foundation for future hybrid knowledge graph construction, where structured metadata and information extracted from biography texts can be combined using persistent researcher identifiers.

The approach is applied to two sources of biographies: IEEE paper-author biographies and ORCID researcher profiles. It extracts three main categories of information: author attributes, education entries, and professional positions. The extraction follows a fixed JSON schema and a carefully designed system prompt. The implementation uses high-performance computing resources to process the full extraction-ready dataset of 390,920 IEEE biography records and 25,759 ORCID researcher profiles. All extracted records preserve their corresponding `dblp_pid`, so the structured output remains linked to existing dblp researcher entities and supports future knowledge graph enrichment.

The extracted information is compared to a manually annotated gold standard of 100 IEEE records and 50 ORCID records using an automated Python comparison pipeline. Evaluation takes place at the field level using precision, recall, F1-score, and accuracy. This also includes field-specific matching strategies and institution-name normalization. The results show strong extraction quality from both sources. For IEEE biographies, the pipeline achieves micro-F1 scores of 0.969 for author-level fields, 0.995 for education fields, and 0.949 for position fields. For ORCID biographies, the corresponding micro-F1 scores are 0.905, 0.966, and 0.913.

Education information is extracted reliably. However, fields related to positions are more challenging because roles, departments, institutions, locations, and time-related information can be ambiguous. The main error patterns include institution-name mismatches, department-boundary ambiguity, location hallucination, and open-ended field errors. Location hallucination remains a significant issue, even with explicit prompt constraints and post-processing validation.

Overall, the findings indicate that schema-driven, in-context LLM extraction is a practical and scalable method for converting academic biography text into structured researcher career information. This thesis provides the extraction, validation, export, and evaluation layer required before future integration into a hybrid scholarly knowledge graph. The resulting data offers a traceable foundation for enriching dblp-linked scholarly knowledge graphs with researcher career information.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Research Context and Use Case . . . 2
- 1.3 Research Problem and Objectives . . . 4
- 1.4 Proposed Approach . . . 5
- 1.5 Thesis Structure . . . 6

2 Foundations and Related Work **7**
- 2.1 Information Extraction from Researcher Biographies . . . 7
- 2.2 Resume, Profile, and Biographical Information Extraction . . . 8
- 2.3 Schema-Guided Extraction with Large Language Models . . . 9
- 2.4 Open-Weight Models and Local LLM Deployment . . . 9
- 2.5 Structured Extraction for Knowledge Graph Enrichment . . . 10
- 2.6 Positioning of This Thesis . . . 11

3 Dataset Analysis and Problem Setting **12**
- 3.1 Data Sources . . . 12
- 3.2 Input Format and Example Records . . . 13
- 3.3 Data Preprocessing . . . 14
- 3.4 Extraction Requirements . . . 16
- 3.5 Summary . . . 17

4 Design and Conception **18**
- 4.1 Design Goals . . . 18
- 4.2 Extraction Schema . . . 19
- 4.3 Prompt Design . . . 21
- 4.4 Model and Deployment Choice . . . 22
- 4.5 Pipeline Architecture . . . 23
- 4.6 Summary . . . 24

5 Implementation **25**
- 5.1 Input Data Preparation and Batch Organization . . . 25
- 5.2 Batch Processing and HPC-based LLM Inference . . . 26
- 5.3 Output Validation, Error Handling, and CSV Export . . . 28
- 5.4 Summary . . . 30

6 Evaluation and Results **32**

6.1 Gold Standard and Evaluation Files 32
6.2 Metrics, Matching, and Normalization 34
6.3 Extraction Results 37
6.4 Error Analysis with Examples 39
6.5 Summary 41

7 Discussion, Conclusion, and Future Work **43**
7.1 Discussion of Findings 43
7.2 Limitations 44
7.3 Future Work 45
7.4 Conclusion 46

A Extraction Prompt **47**

B Python Packages and Standard Modules **54**

References **55**

List of Figures

3.1 Distribution of extraction-ready biography lengths in IEEE and ORCID, measured in characters. Extreme outliers are omitted for readability; minimum and maximum values are reported in Table 3.2. . . 16

4.1 High-level architecture of the extraction pipeline. 23

List of Tables

3.1 Preprocessing summary for the IEEE and ORCID biography datasets. 15

3.2 Descriptive statistics of extraction-ready biography lengths in characters. 15

4.1 Overview of the extraction schema. 20

5.1 Approximate wall-clock runtime for full dataset extraction. 27

6.1 Simplified example of a manual gold standard annotation. 33

6.2 Main files used in the evaluation workflow. 34

6.3 Simplified example of manual gold standard and LLM output comparison for an education record. 34

6.4 Example of TP, TN, FP, and FN outcomes for the field `location`. . . 35

6.5 Examples of string comparison and normalization issues. 36

6.6 Micro-averaged extraction results by source and entity group. 37

6.7 Author entity field comparison in IEEE and ORCID. 38

6.8 Education entity field comparison in IEEE and ORCID. 38

6.9 Position entity field comparison in IEEE and ORCID. 38

6.10 Selected diagnostic error categories in IEEE and ORCID. Counts refer to selected FP and FN observations after row alignment; multi-entry merging is counted from unmatched education or position entries. . . 39

6.11 Examples of recurring extraction errors. 41

B.1 Python packages and standard modules used in the implementation. . 54

1. Introduction

1.1 Motivation

Scholarly information systems hold large amounts of structured metadata about academic publications, authors, venues, and affiliations. In computer science, dblp is one of the key bibliographic infrastructures. It provides curated publication metadata along with persistent author identifiers [ABB+24]. Such infrastructures are crucial for finding publications, analyzing research output, identifying authors, and connecting scholarly entities.

However, publication metadata alone does not provide a complete view of a researcher's academic profile. Many important details about a researcher's career, including educational background, past and present positions, research interests, awards, memberships, projects, scholarly output, and professional activities, often go unrepresented in structured bibliographic records. This information is vital for tasks like researcher profiling, expertise search, collaboration recommendations, academic mobility analysis, and studies on the science of science.

Much of this researcher-focused information can be found in a biography text. Author biographies linked to publications and researcher-maintained profiles often cover degrees, institutions, departments, career stages, research areas, awards, memberships, and academic achievements using natural language. While these descriptions add depth to structured metadata, they are difficult to use directly because they are unstructured, varied, and not easily machine-readable.

This thesis focuses on two sources of biography connected to dblp researcher identifiers: IEEE author biographies and ORCID researcher profiles. The IEEE biographies are linked to publications and typically follow a concise academic style. ORCID profiles, in contrast, are maintained by researchers and vary significantly in length, completeness, and wording [ORC25], [dbl25]. Both datasets contain a `dblp_pid`, which enables a clear connection between each biography record and an existing researcher entity in dblp. In this thesis, the two sources are processed separately; there is no linking or merging of entities between IEEE and ORCID records.

The primary goal of this thesis is to convert unstructured academic biography text into structured information about researchers' careers. The focus is on the extraction layer of creating a hybrid knowledge graph rather than on building the final knowledge graph itself. The pipeline produces schema-conforming JSON records from biography text while carrying the `dblp_pid` from the input record through to the output and exported CSV files. This keeps the extracted career information connected to the respective researcher and lays a structured foundation for future knowledge graph integration.

1.2 Research Context and Use Case

The extraction task addressed in this thesis can be understood as the transformation of a free-text researcher biography into a structured JSON record. Each input record contains the biography text and the identifiers needed to track it through the pipeline. The `dblp_pid` is used as the main researcher-level key. IEEE records additionally contain a `doi`, while ORCID records contain an `orcid` identifier. These source-specific identifiers are used to distinguish the data source and to match the extracted output with the corresponding gold standard records during evaluation.

The output is a schema-conformant JSON object that organizes the extracted information into researcher-level attributes, education entries, professional positions, scholarly metrics, and additional career-related fields. The following anonymized example illustrates the task. It does not refer to a real researcher, but reflects the type of information commonly found in IEEE and ORCID biography records.

> *Researcher A was born in City Z, Country Z, in 1985. She received the M.Sc. degree in Computer Science from University X, City X, Country X, in 2000. Since 2021, she has been an Associate Professor in the Department of CS at University Y, City Y, Country Y. Her research interests include knowledge graphs, information extraction, and natural language processing. She has published more than 40 papers in top-tier NLP/ML/AI conferences and journals. She received the Best Paper Award in 2022 and is a Senior Member of IEEE. She served as a reviewer for several international conferences and delivered an invited tutorial on knowledge graph construction.*

From this biography, the extraction pipeline produces a structured JSON output. A simplified example is shown in Listing 1.1. Fields not supported by the biography text are included in the output and given null values or empty lists as needed.

Listing 1.1: Simplified anonymized example of the structured JSON extraction output.

```
{
  "dblp_pid": "example/123",
  "entities": {
    "full_name": "Researcher A",
    "personal": {
      "birth_year": 1985,
      "birth_location": "City Z, Country Z",
      "nationality": "Country Z"
    },
    "education": [
      {
        "degree": "M.Sc. degree",
        "field": "Computer Science",
        "university": "University X",
        "location": "City X, Country X",
        "edu_year": 2000,
        "is_current": false
      }
    ],
```

```
    "positions": [
      {
        "role": "Associate Professor",
        "department": "Department of CS",
        "institution": "University Y",
        "location": "City Y, Country Y",
        "start_year": 2021,
        "end_year": null,
        "is_current": true,
        "description": null
      }
    ],
    "scholarly_metrics": [
      {
        "metric_type": "publications",
        "count": 40,
        "description": "published more than 40 papers in top-
           tier NLP/ML/AI conferences and journals"
      }
    ],
    "research_areas": [
      "knowledge graphs",
      "information extraction",
      "natural language processing"
    ],
    "awards": [
      {
        "name": "Best Paper Award",
        "year": 2022
      }
    ],
    "projects": null,
    "memberships": [
      "Senior Member of IEEE"
    ],
    "activities": [
      "served as a reviewer for several international
         conferences"
    ],
    "raw_facts": [
      "delivered an invited tutorial on knowledge graph
         construction"
    ]
  }
}
```


The value Country Z is used here only as a placeholder. In the actual extraction task, nationality is extracted if it is clearly stated in the biography or can be inferred from a birth location according to the extraction rules. This is the only permitted inference rule in the extraction task. Apart from nationality inferred from an explicitly stated birth location, all other fields are extracted only when they are directly supported by the biography text.

This example shows that the task is more than just recognizing names. The LLM is told not only to identify strings like university names, degrees, or job titles, but also to place these strings in the appropriate schema fields. The schema structure keeps the connections between related fields. For instance, an education entry combines the degree, university, location, and graduation year, while a position entry combines the role, department, institution, location, and start year.

The pipeline does not produce a finished knowledge graph. Instead, it transforms biography text into structured JSON records that keep the researcher identifier intact and organize the extracted information in a consistent, predictable way. Those records are not the end product; they are an intermediate step. Once validated and cleaned, they can serve as input for building a hybrid knowledge graph that brings together structured bibliographic metadata and the richer career detail that biography text contains.

1.3 Research Problem and Objectives

The sections above show that academic biographies contain a meaningful amount of researcher-centered information that is worth capturing. But before that information can support any kind of analysis or feed into a knowledge graph, it needs to be organized into a structured form. The central problem addressed in this thesis is therefore not the availability of biography text itself, but the reliable extraction of structured career information from heterogeneous biography narratives.

This task is more challenging than it may initially appear. A single sentence in a biography can mention a degree, a university, a location, a year, a current position, a department, and a research interest all at once. An extraction system working on text like that has to do more than identify isolated keywords. It has to produce output that accurately reflects both what the biography says and how the different pieces of information relate to each other. It also must represent missing information clearly, resist the temptation to fill in gaps with plausible-sounding guesses, and keep every extracted record tied to the `dblp_pid` it came from.

Compared to more standard information extraction tasks, this setting comes with a few extra complications. The system has to handle biography texts that vary considerably in style and detail. It has to support researchers who have multiple degrees and multiple positions rather than just one of each. It has to preserve the `dblp_pid` throughout so that traceability is never lost. It has to avoid drawing conclusions that the biography does not actually support. And the output has to conform to a defined schema rather than just returning whatever the model finds relevant. Put together, these demands make the task meaningfully more involved than extracting a name or a date from a sentence. With this in mind, the main research question of this thesis is:

> *How can a schema-driven, prompt-based large language model pipeline extract structured researcher career information from heterogeneous academic biography text at scale, without task-specific model fine-tuning, and in a form suitable as a basis for future hybrid knowledge graph construction?*

To address this research question, the thesis follows five research objectives.

1. **Define a unified extraction schema.** The goal is to create a schema that clearly represents researcher-level attributes, education entries, professional positions, scholarly metrics, and additional career-related information in a consistent way.
2. **Design a schema-driven prompt-based extraction approach.** The aim is to develop a prompt-based strategy that allows a large language model to produce structured JSON output from academic biography text without specific fine-tuning.
3. **Implement a scalable extraction pipeline.** The objective is to establish a pipeline for processing IEEE author biographies and ORCID researcher profiles while retaining researcher identifiers and source-specific metadata for traceability.
4. **Evaluate the extraction quality.** This involves assessing the extracted author-level, education, and position information against manually annotated gold standard data using precision, recall, F1-score, and error analysis.
5. **Analyze the suitability for future knowledge graph construction.** The fifth objective focuses on determining whether the extracted JSON and CSV outputs provide a traceable and schema-consistent basis for future hybrid knowledge graph creation.

Together, these objectives outline the thesis's scope. The work focuses on extracting, validating, and evaluating structured researcher career information from biography text. The resulting JSON and CSV outputs define the structured extraction layer discussed in this thesis.

1.4 Proposed Approach

To address the research problem, this thesis proposes a schema-driven extraction pipeline that turns academic biography text into structured researcher career information. The work moves through five stages: preprocessing the input data, extracting information using a prompt-based large language model, validating the output, exporting it into tabular form, and comparing the results against manually annotated gold standard data.

The pipeline starts with the IEEE and ORCID biography datasets. Both provide free-text biographies alongside researcher identifiers, but they come from different places and are structured differently. IEEE biographies are connected to publication records, while ORCID biographies are written and kept up to date by the researchers themselves. The two datasets are processed in separate runs, and the identifiers introduced in Section 1.2 are carried through at every stage so that each extracted record can always be linked back to where it came from.

After preprocessing, each biography is put into a consistent input format and sent to a self-hosted open-weight instruction model, LLaMA 3.1 SauerkrautLM 70B Instruct, along with a schema-driven extraction prompt. The same schema and prompt logic

are used for both IEEE and ORCID biographies. The prompt instructs the model to return valid JSON and to populate a fixed set of fields covering researcher attributes, education records, professional positions, scholarly metrics, and other career-related information. If the biography does not mention a particular field, that field stays in the output but is set to null or an empty list rather than being dropped.

The JSON output is then validated before being converted into tabular form. Because the schema includes nested education and position entries, the output is flattened into three CSV tables: one at the author level, one for education entries, and one for professional positions. Splitting the data this way makes it much easier to inspect individual fields, spot issues, and run comparisons during evaluation.

Finally, the extracted information is compared against manually annotated records. The evaluation works at the field level and reports precision, recall, and F1-score alongside diagnostic error patterns that help explain where the pipeline does well and where it falls short.

Taken together, the thesis delivers a schema-driven pipeline that moves from raw biography text to structured career data and sets the stage for future integration with dblp-linked scholarly data.

1.5 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 presents the foundations and related work relevant to this thesis. It focuses on extracting information from text, structured extraction with large language models, resume and profile extraction, scholarly information extraction, and enriching knowledge graphs. The chapter positions this thesis in relation to existing work and highlights the specific research gap addressed by the proposed approach.

Chapter 3 analyzes the problem setting and datasets. It describes the IEEE and ORCID biography sources, explains the preprocessing steps applied to both datasets, illustrates the structure of the input data, and derives the main extraction requirements.

Chapter 4 outlines the design and conception of the extraction pipeline. It introduces the design goals, the extraction schema, the prompt design strategy, the model and deployment choice, and the pipeline layout.

Chapter 5 describes how the pipeline is implemented. It explains how to prepare input records, manage records using SQLite, process batches with SLURM, perform large language model inference with vLLM, validate output, and export it into author, education, and position tables.

Chapter 6 details the evaluation methodology and results. It describes how to create the gold standard, the field-level matching strategy, the evaluation metrics, comparative result tables for IEEE and ORCID, and the analysis of diagnostic errors.

Chapter 7 discusses the thesis findings. It summarizes the main findings, discusses limitations, and outlines future work directions, including potential integration of the extracted outputs into hybrid scholarly knowledge graphs.

2. Foundations and Related Work

This chapter outlines the foundations and related work that underlie the extraction pipeline developed in this thesis. Rather than treating information extraction, large language models, and knowledge graphs as separate topics, this chapter emphasizes how these areas connect to the specific task of extracting structured researcher career information from academic biography text. It begins by discussing information extraction from researcher biographies, followed by a review of resume, profile, and biographical information extraction. The chapter then covers schema-guided extraction with large language models, the role of open-weight model deployment, and the link between structured extraction and future knowledge graph enrichment. Finally, the chapter positions this thesis in relation to existing work.

2.1 Information Extraction from Researcher Biographies

Information extraction aims to transform information expressed in natural language into a structured representation that can be stored, queried, analyzed, and evaluated computationally [XCP+24]. In this thesis, the input is a short academic biography rather than a document with clearly marked sections. Such biographies are compact and information-dense: a single paragraph may mention a degree, university, graduation year, current role, department, institution, location, research interests, awards, memberships, and publication counts.

The task is therefore more complex than recognizing isolated named entities. A conventional named entity recognition system may identify organization names, locations, dates, and titles, but it does not necessarily produce the nested structure needed for this thesis. For example, a degree, university, location, and graduation year must be grouped into one education entry, while a role, department, institution, location, and start year must be grouped into one position entry. The extraction task, therefore, combines entity recognition, attribute extraction, relation-like grouping, and schema completion.

Researcher biographies also vary strongly in wording. Education may be described through phrases such as “received the Ph.D. degree”, “obtained his doctorate”, or “holds a master’s degree”. Professional roles may appear as current appointments, previous positions, visiting roles, editorial roles, or fellowships. Missing information must be represented explicitly instead of being inferred from external knowledge. These requirements motivate the use of a schema-driven extraction design in which all required fields are present, unsupported fields are left empty, and the output can be validated before evaluation.

2.2 Resume, Profile, and Biographical Information Extraction

Resume and curriculum vitae extraction are closely related to this thesis because they also focus on career-related information such as education, employment history, skills, institutions, and dates. Resume extraction work supports the selection of education and professional positions as core fields in the extraction schema. However, resumes and academic biographies differ in structure. Resumes often contain headings, lists, and semi-structured sections, whereas IEEE and ORCID biographies are usually written as free-form narrative paragraphs. This makes academic biography extraction less dependent on layout cues and more dependent on sentence-level interpretation.

Li et al. [LSZL21] propose a BERT-BiLSTM-CRF method for resume information extraction. Their work shows that education and professional experience are central targets in career-document extraction and illustrates how supervised sequence-labeling models can be used for structured career information extraction. Such models are effective when annotated training data and stable label definitions are available. The setting of this thesis is different: the manually annotated data is used for evaluation rather than model training, and the input biographies are shorter and less structured than resumes. This setting motivates the use of prompt-based extraction without task-specific fine-tuning.

Barducci et al. [BILG+22] present an end-to-end information extraction framework for Italian resumes. Their work illustrates that career-document extraction is usually a pipeline task involving document processing, entity extraction, and post-processing. This pipeline perspective is reflected in the present thesis through preprocessing, LLM-based extraction, validation, and tabular export. The input setting is different, however, because this thesis works with academic biographies linked to dblp researcher identifiers rather than resumes submitted for recruitment.

Gan and Mori [GM23] provide useful evidence for the role of prompt design in career-related information extraction. Their work applies prompt-based learning to resume information extraction and shows that domain-specific prompt components can substantially improve performance. In their experiments, the manually designed knowledgeable verbalizer improved F1 from 54.96 to 63.65 in the 25-shot setting, from 59.72 to 76.53 in the 50-shot setting, and from 66.46 to 76.72 in the 100-shot setting. These results support the design choice in this thesis to use a task-specific extraction prompt with explicit field definitions and extraction rules instead of a generic extraction instruction. Although their task focuses on resume sentence classification rather than nested JSON extraction from academic biographies, the results show that prompt formulation is important for extracting structured career information.

A very recent and closely related study by Winkle, Stede, and Kreutel [WSK26] examines generative information extraction from biographical sources using open-source large language models. Their work is relevant because it also addresses biography-like texts and evaluates LLM-based extraction in a structured setting. Since the study appeared only recently, it is treated here as contemporary related work rather than as the original basis for the design of this thesis.

Together, resume and biographical extraction work shows that career-related information can be extracted from text, but it also highlights the limitations of existing

approaches for this thesis. Resume extraction typically assumes more regular document structures, while biographical extraction is closer to the input style of this thesis but remains less explored in the specific setting of large-scale academic biographies linked to persistent dblp researcher identifiers. This motivates the need for a schema-driven extraction approach tailored to researcher biographies.

2.3 Schema-Guided Extraction with Large Language Models

Large language models make it possible to treat information extraction as a generative task. Instead of building a separate supervised model for each set of labels, the extraction task can be described using instructions that define the target fields, output format, and rules for handling missing or uncertain information. Xu et al. [XCP+24] explain this trend as part of a larger movement toward generative information extraction. Here, LLMs create structured representations from natural language text.

This idea is central to this thesis. The goal is not simply to gather a flat list of entities but to produce a nested JSON object. The schema includes researcher-level attributes, education entries, position entries, scholarly metrics, and more career-related fields. The model needs to fill this structure using only the information in the biography and must leave unsupported fields empty. Thus, the schema acts as both extraction instructions and a way to ensure validation, CSV export, and evaluation.

Dagdelen et al. [DDL+24] strongly support this design choice. They demonstrate that structured scientific information can be extracted from text using large language models and that this output can be organized in structured formats such as JSON. In their host-dopant extraction task, the Llama-2 model with the JSON schema achieved the best performance among six LLM-based versions, reaching an F1-score of 0.821. This finding supports using schema-compliant JSON output in this thesis, emphasizing the need to maintain connections between related fields like degree, university, and year, as well as role, institution, and department.

Schema guidance also helps address common shortcomings of LLM-based extraction. Zhu et al. [ZWC+24] discuss how LLMs contribute to knowledge graph creation and reasoning. They highlight challenges like hallucination, schema compliance, and consistency. These challenges directly inform the stringent extraction rules employed in this thesis. This includes clear handling of null values, limitations on unsupported location inferences, and validation before evaluation.

2.4 Open-Weight Models and Local LLM Deployment

To extract information from hundreds of thousands of biography records, a model setup must run in a controlled and repeatable environment. This thesis chooses a self-hosted open-weight instruction model instead of depending on a closed external API. Local deployment allows for batch processing, consistent generation settings, and direct control over the extraction process. These factors are all important for managing large biographical collections.

The production extraction process uses LLaMA 3.1 SauerkrautLM 70B Instruct. The SauerkrautLM model card describes it as an instruction-tuned model built on Meta

LLaMA 3.1 70B Instruct and fine-tuned on German-English data using Spectrum Fine-Tuning [VAG24]. Meta describes the LLaMA 3.1 model family as a group of pre-trained and instruction-tuned models aimed at multilingual text generation and long-context applications [Met24]. These characteristics fit the practical needs of this thesis, where biography records can vary in wording, style, and source, and need to be processed in a local batch environment.

The choice of model is part of the implementation rather than a separate comparison of models. During earlier development, several instruction models were evaluated using the extraction prompt. LLaMA 3.1 SauerkrautLM 70B Instruct was chosen for the production run because it delivered the best schema-compliant outputs in preliminary tests and could be deployed in the self-hosted inference environment. Chapter 6 evaluates the extraction pipeline's quality based on this selected model, rather than comparing all available large language models.

This choice of deployment relates to the overall design of the thesis. The goal is not only to generate individual answers from a language model but to handle large sets of researcher biographies in a repeatable way. The selected model is used along with a fixed extraction prompt, consistent generation settings, JSON validation, and tabular export. Together, these elements ensure that LLM output is suitable for systematic evaluation and future structured integration into knowledge graphs.

2.5 Structured Extraction for Knowledge Graph Enrichment

Knowledge graph construction and enhancement are important to this thesis because the extracted information aims to provide a structured foundation for future integration with scholarly data. A knowledge graph represents entities and relationships in a form that machines can understand. In academia, entities may include researchers, publications, institutions, venues, topics, and affiliations. dblp provides curated bibliographic metadata and persistent researcher IDs, making it a suitable framework for linking publication data with additional researcher-related information [ABB+24].

This thesis emphasizes the extraction layer prior to integration into a graph. Complete knowledge graph construction would require additional steps such as modeling ontologies, linking entities, creating URIs, converting to RDF, and loading the graph. The contribution here is the initial step of turning biography text into structured, traceable records. Constructing the full graph will be addressed as future work, as discussed in Chapter 7.

Zhu et al. [ZWC+24] explain how LLMs can assist in building and reasoning about knowledge graphs. Their work is relevant because it confirms that LLMs can handle tasks like entity extraction, relationship extraction, and graph enhancement. Their insights into hallucination and consistency issues support the careful design of this thesis. Since the extracted information could later be used in a graph, the extraction system must avoid unsupported inferences and produce output that is valid.

The JSON and CSV outputs in this thesis act as a middle step between biography text and future graph integration. For example, an education entry that includes a researcher ID, degree, university, year, and location could later transform into graph nodes and relationships. Likewise, a position entry with a role, department,

institution, start year, and current status flag could help outline researchers' career paths in a scholarly knowledge graph. This shows how clear and consistent the extraction output can be to support future graph integration.

2.6 Positioning of This Thesis

The work reviewed in this chapter supports the main design decisions made in this thesis. Research on resume extraction motivates treating education and professional positions as the central career information fields. Studies on prompt-based resume extraction show that investing in task-specific prompt design can make a real difference to extraction quality, particularly when training data is limited. Work on generative information extraction from biographical sources confirms that LLM prompting can be applied to biography-like texts, but also makes clear that recall and evaluation are not straightforward problems to solve. Schema-guided scientific information extraction supports the decision to use structured JSON output, and work on LLMs for knowledge graph construction highlights the importance of handling hallucination carefully, keeping the output schema consistent, and maintaining traceability throughout.

What this thesis does is bring these threads together in a setting that existing work does not fully address. The input data consists of IEEE author biographies and ORCID researcher profiles, both linked to `dblp_pid` identifiers. The output schema is built around researcher-centered career information and covers author attributes, education records, professional positions, scholarly metrics, and a range of additional career-related fields. The extraction is carried out without any task-specific fine-tuning, and the results are evaluated at the field level against manually annotated gold standard data.

The focus of the thesis is therefore on the extraction and evaluation layer that needs to exist before any knowledge graph integration can happen. It does not assume that the biography text arrives in a structured form, and it does not depend on supervised training data for the extraction model. Instead, it asks whether a schema-driven, prompt-based LLM pipeline can produce traceable, schema-consistent career records from academic biographies that vary widely in style, length, and the information they contain.

3. Dataset Analysis and Problem Setting

This chapter explains the data used in this thesis and what the extraction pipeline must handle. It introduces the IEEE and ORCID biography sources, outlines the structure of the input records, and summarizes the preprocessing steps for preparing the biographies for extraction. The chapter also points out the main requirements that come from the input data. These requirements form the basis for the design decisions discussed in Chapter 4.

3.1 Data Sources

This thesis uses two sources for the researcher biography text: IEEE biographies and ORCID biographies. Both datasets include links to dblp researcher identifiers. The `dblp_pid` is preserved throughout preprocessing, extraction, and evaluation, while `doi` and `orcid` are used as source-specific identifiers for IEEE and ORCID records. The dblp Knowledge Graph provides a semantic representation of dblp bibliographic data and offers access via RDF dumps and a SPARQL endpoint [ABB+24].

The IEEE dataset comes from biographies linked to IEEE journal articles indexed by dblp from 2012 to 2025. Each raw IEEE record corresponds to a paper-author-biography entry. A researcher may appear multiple times if they have authored several IEEE publications that include biography text. IEEE records contain biography text along with publication-related metadata like `doi` and affiliation information.

The ORCID dataset is based on researcher-maintained ORCID profiles and reflects the ORCID Public Data File 2025 [ORC25]. The records are connected to dblp identifiers through the dblp public XML release [dbl25]. ORCID records contain biography text along with the `orcid` identifier and may also include structured profile information such as education and employment details.

Although both datasets have extra metadata, the extraction task in this thesis focuses only on the free-text biography as model input. IEEE affiliation fields and ORCID education or employment fields are kept in the data, but they are not passed to the model during extraction. This keeps the task centered on biography text rather than structured fields.

The two sources are processed separately. There is no linking or merging between IEEE and ORCID records. The `dblp_pid` serves as the key at the researcher level. The `doi` for IEEE records and the `orcid` for ORCID records are used as specific identifiers for each source.

The next section shows the structure of the raw records and the simplified input representation used for LLM extraction.

3.2 Input Format and Example Records

The raw IEEE and ORCID records contain biography text along with source-specific identifiers and additional metadata. Before extraction, each selected biography is converted into the simplified input format required by the extraction script. The following examples first display the raw record structure for both sources and then present the simplified input record used for LLM extraction.

Listing 3.1 shows a simplified IEEE-style raw record. The record includes a `doi`, a `dblp_pid`, a biography list, and affiliation information. The `doi` identifies the publication record, while `dblp_pid` serves as the researcher-level key.

Listing 3.1: Simplified anonymized IEEE-style raw record.

```
{
  "doi": "10.xxxx/example.doi",
  "dblp_pid": "example/ieee-001",
  "biographies": [
    "Researcher A was born in City X, Country X, in 1974. He
       received the B.S. degree from University X in 1998 and
       the Ph.D. degree from Institute Y in 2004. His research
        interests include optical instruments and opto-
       mechanical systems."
  ],
  "affiliations": [
    "Department of Example Engineering, University Z, City Z,
        Country Z"
  ]
}
```

Listing 3.2 shows a simplified ORCID-style raw record. This record includes an `orcid`, a `dblp_pid`, a biography list, and may also include structured education and employment details.

Listing 3.2: Simplified anonymized ORCID-style raw record.

```
{
  "orcid": "0000-0000-0000-0000",
  "dblp_pid": "example/orcid-001",
  "biographies": [
    "Researcher B has been working as an Associate Professor
       at University Y since 2021. Before that, he was a
       researcher at Institute X from 2016 to 2020. His
       research interests include natural language processing.
        He has published more than 40 papers in top-tier
       conferences and journals."
  ],
  "educations": [
    {
      "org_name": "University Y, Department of Computer
         Science",
      "org_country": "XX",
      "org_city": "City Y",
```

```
      "start_date": "2012-09-01",
      "end_date": "2016-06-12",
      "role": "Ph.D."
    }
  ],
  "employments": [
    {
      "org_name": "University Y, Department of Computer
         Science",
      "org_country": "XX",
      "org_city": "City Y",
      "start_date": "2021-01-01",
      "end_date": null,
      "role": "Associate Professor"
    }
  ]
}
```


These examples show that the raw records contain more information than just the biography text. For extraction, however, the model input is limited to the `biographies` field. The IEEE affiliation metadata and ORCID education or employment entries remain in the dataset, but the model does not use them as textual evidence.

Listing 3.3 shows a simplified input record used for LLM extraction. As IEEE and ORCID were processed separately, the data source itself was not included as a field in the model input.

Listing 3.3: Simplified input record used for LLM extraction.


```
{
  "dblp_pid": "example/orcid-001",
  "orcid": "0000-0000-0000-0000",
  "biography": "Researcher B has been working as an Associate
      Professor at University Y since 2021. Before that, he
     was a researcher at Institute X from 2016 to 2020. His
     research interests include natural language processing."
}
```


For IEEE records, the same simplified structure is applied, but the source-specific identifier is `doi` instead of `orcid`. This input format allows the same prompt and structure to be used for both biography sources while keeping the extracted output connected to the original record.

3.3 Data Preprocessing

The preprocessing step converts the raw IEEE and ORCID records into extraction-ready biography records. The aim is to keep one usable biography text per researcher-level record while preserving the identifiers needed for extraction and evaluation.

For the IEEE dataset, the raw file contains 1,155,481 paper-author-biography records. Since IEEE biographies are publication-associated, the same researcher may occur in several records. The `dblp_pid` was therefore used as the researcher-level grouping key.

For each `dblp_pid`, the longest available biography was selected as the representative biography. This reduced the IEEE dataset to 390,928 researcher-level biography records. Afterwards, biographies shorter than 50 characters were removed because they did not provide enough textual information for meaningful extraction. This removed eight records and resulted in 390,920 extraction-ready IEEE biographies.

For the ORCID dataset, the raw file contains 26,774 records. First, 12 records with null or empty biography fields were removed. Duplicate `dblp_pid` cases were then examined because more than one ORCID record may be linked to the same dblp researcher. After duplicate resolution, 26,752 records remained. Four conflicting cases were excluded because the biography content appeared to describe a different researcher than the one indicated by the associated `dblp_pid`. Finally, the same 50-character minimum length threshold was applied, removing 989 short biographies. The final ORCID extraction-ready dataset contains 25,759 biographies.

Table 3.1: Preprocessing summary for the IEEE and ORCID biography datasets.

Preprocessing step	**IEEE**	**ORCID**
Raw records	1,155,481	26,774
Null or empty biographies removed	0	12
After deduplication or duplicate resolution	390,928	26,752
Conflicting duplicate cases excluded	–	4
Biographies shorter than 50 characters excluded	8	989
Final extraction-ready records	**390,920**	**25,759**

To further characterize the final extraction-ready biographies, descriptive statistics were computed for biography length after preprocessing and after applying the final filtering rules. The statistics are measured in characters and summarize the length distribution of the biography texts. Biography length is relevant because it affects both the amount of information available for extraction and the amount of text processed by the model. The mean biography length is 597.71 characters for IEEE and 1101.78 characters for ORCID.

Table 3.2: Descriptive statistics of extraction-ready biography lengths in characters.

Dataset	**Records**	**Mean**	**Q1**	**Median**	**Q3**	**Min**	**Max**
IEEE	390,920	597.71	374	503	715	50	8,225
ORCID	25,759	1101.78	475	852	1,435	50	5,000

Figure 3.1 visualizes the biography-length distribution using a boxplot. The boxplot confirms that ORCID biographies are generally longer and more variable than IEEE biographies. The median length is 503 characters for IEEE and 852 characters for ORCID, while the wider ORCID box indicates a larger spread in the middle 50% of biographies.

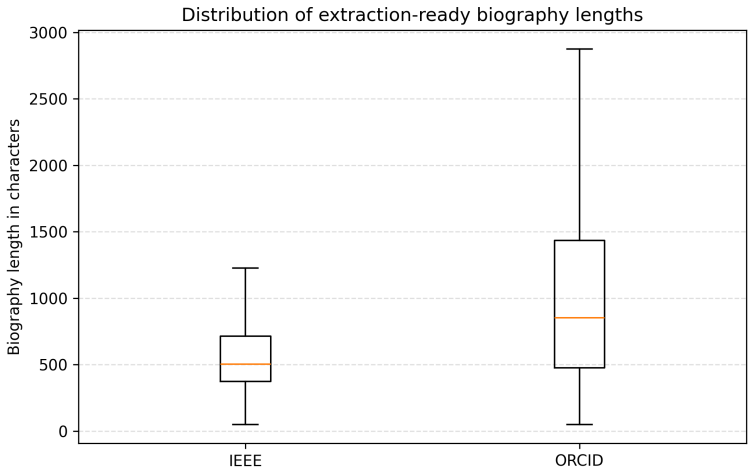


Figure 3.1: Distribution of extraction-ready biography lengths in IEEE and ORCID, measured in characters. Extreme outliers are omitted for readability; minimum and maximum values are reported in Table 3.2.

Across both sources, the final extraction-ready dataset contains 416,679 biography records. These records form the input collection for the LLM extraction pipeline described in the following chapters.

3.4 Extraction Requirements

The dataset structure and preprocessing decisions define the main requirements for the extraction pipeline.

First, the extracted information must remain connected to the corresponding `dblp_pid`. This identifier is used as the researcher-level key during preprocessing, extraction, and evaluation.

Second, the pipeline requires one common extraction schema for both IEEE and ORCID biographies. Although the two sources are processed independently, the output must follow the same structure so that the extracted records can be validated, exported, and evaluated consistently.

Third, the schema needs to support multiple career entries per researcher. Biographies rarely describe a single degree or a single job; most include several education entries, multiple positions, a handful of research areas, and various awards, memberships, or activities accumulated over a career. A doctoral degree and an earlier research role, for instance, belong in separate records under education and positions, respectively, not collapsed into a single text field where the distinction between them is lost.

Fourth, missing information needed to be handled consistently. Not every biography mentions every field the schema defines. If a biography lists a degree but says nothing about when it was awarded, the `edu_year` field should simply be left empty rather than filled in with a guess drawn from somewhere else. This was a deliberate choice. The task is to extract what the biography actually says, not to reconstruct details that are not there.

Finally, the output must be suitable for automated processing. The LLM output must be valid JSON so that it can be checked, cleaned, and flattened into author-level,

education, and position tables. These tables are required for the field-level evaluation described in Chapter 6.

These requirements motivate the schema-driven prompt design and pipeline architecture presented in Chapter 4.

3.5 Summary

This chapter explained the biography datasets used in this thesis and outlined how they are prepared for extraction. The IEEE and ORCID sources both contain free-text biographies linked to `dblp_pid` identifiers. Additional metadata is kept in the datasets but is not used as input for the model. The extraction task focuses solely on biography text, allowing for evaluation based on information extracted from natural language biographies.

The preprocessing step turned the raw records into biography records that were ready for extraction. For IEEE, publication-associated records were grouped by `dblp_pid`, and the longest biography was selected for each researcher. After filtering by length, the final IEEE dataset contains 390,920 biographies. For ORCID, empty biographies, duplicate cases, conflicting records, and very short biographies were removed. This resulted in 25,759 extraction-ready biographies. The total collection from both sources includes 416,679 biography records.

This chapter also describes the main requirements for the extraction pipeline. The pipeline must preserve the `dblp_pid`, use a common schema for both sources, support multiple career entries, handle missing information explicitly, and produce valid JSON output for later validation and evaluation. These requirements form the basis for the schema design and pipeline architecture discussed in Chapter 4.

4. Design and Conception

This chapter explains the design and conception of the extraction pipeline developed in this thesis. Based on the dataset analysis and extraction requirements outlined in Chapter 3, this chapter describes how the proposed solution is structured to extract researcher career information from IEEE and ORCID biographies. The chapter focuses on the conceptual design of the extraction approach, which includes design goals, schema definition, prompt strategy, system architecture, and model selection. Specific implementation details will be discussed in Chapter 5.

4.1 Design Goals

The design of the extraction pipeline follows from the requirements identified in Chapter 3. The main task is to turn unstructured biography text into structured researcher career information while keeping the connection to the corresponding `dblp_pid`. This means that the pipeline has to handle natural language input flexibly, but still produce output that is consistent enough to parse, validate, export, and evaluate.

The first goal was to use the same extraction logic for both data sources. IEEE and ORCID biographies come from different sources and differ in style, length, and level of detail. However, building two separate extraction systems would make the implementation more complex and would also make the results harder to compare. For this reason, the same schema and prompt logic are used for both sources, while IEEE and ORCID are processed in separate runs.

The second goal was to keep the output structure consistent across all records. Large language models can otherwise return slightly different formats, for example, by changing field names, adding explanatory text, or nesting information differently. A fixed schema helps avoid this problem by defining which fields should appear, how they should be named, and how repeated information should be organized. This consistency is necessary for automatic parsing, validation, CSV export, and evaluation.

The third goal was to represent academic careers without flattening important details. A researcher may have several degrees, multiple professional positions, different research areas, awards, memberships, or activities. If all of this information were stored in one row or one text field, important relationships between related facts would be lost. Therefore, education entries, position entries, and list-based author fields are kept as separate structured elements.

The fourth goal was to keep the extraction grounded in the biography text. If a value is not stated in the biography, the corresponding field should remain empty instead

of being inferred from external knowledge. This is important because the thesis focuses on extracting information from text, not on completing missing biographical information. Since large language models can produce plausible but unsupported information [STA+24], the prompt and validation logic were designed to reduce unsupported inference.

The fifth goal was to make the pipeline scalable. The final extraction-ready dataset contains 416,679 biography records, so the pipeline needed to support large-scale batch processing. This required batch-based execution, separate output and log files, and resume logic so that interrupted jobs could continue without repeating records that had already been processed.

The final goal was to make the output usable for evaluation. The extracted data needed to follow a stable structure so that it could be compared with the manually annotated gold standard records. This required generating valid JSON, managing missing values uniformly, separating author-level details from education and position information, and maintaining the identifiers that connect each output record to its original biography.

Together, these goals define what the pipeline is designed to do. It takes biography text as input and produces structured JSON and CSV outputs. These outputs form the extraction and evaluation layer of the thesis and can later be used as a basis for integration with dblp-linked scholarly data.

4.2 Extraction Schema

The extraction schema defines the structure that the language model must generate for each biography. It is included in the extraction prompt to ensure that the model produces structured JSON output instead of free-form text. The schema represents researcher career information in a form that can be validated, cleaned, flattened into CSV files, and evaluated at the field level.

The schema is organized into three output groups: author records, education records, and position records. The author group stores biography-level information about the researcher, while the education and position groups allow multiple structured entries for degrees and career positions. Table 4.1 summarizes the main fields and the purpose of each output group.

Table 4.1: Overview of the extraction schema.

Output group	Fields and purpose
Author	**Main fields:** `full_name`, `birth_year`, `birth_location`, `nationality`, `research_areas`, `awards`, `memberships`, `activities`, `projects`, `scholarly_metrics`, `raw_facts`. These fields describe biography-level information about the researcher, including personal details, research interests, achievements, memberships, academic activities, projects, and other relevant profile facts.
Education	**Main fields:** `degree`, `field`, `university`, `location`, `edu_year`, `is_current`. These fields represent one or more education entries. Each entry describes a degree or study period, including the degree type, field of study, institution, location, year, and whether the education is ongoing.
Position	**Main fields:** `role`, `department`, `institution`, `location`, `start_year`, `end_year`, `is_current`, `description`. These fields represent one or more career positions. Each entry describes a role held by the researcher, including department, institution, location, time period, current-status information, and additional role-related description when needed.

Fields that are not explicitly stated in the biography are represented as `null`. Some author-level fields, such as `research_areas`, `awards`, `memberships`, `activities`, `projects`, and `raw_facts`, can contain multiple values. Education and position records can also contain multiple entries, because one researcher may have several degrees or several career positions.

Author records capture information that describes the researcher as a whole. This includes identity-related fields, such as `full_name`, `birth_year`, `birth_location`, and `nationality`, as well as broader career-related information such as research areas, awards, memberships, activities, projects, scholarly metrics, and raw facts. These fields are not tied to a single education or position entry.

Education records represent study-related information mentioned in the biography. Each record keeps the `degree`, `field`, `university`, `location`, and `edu_year` together when these values are stated. The `is_current` field is used as a status indicator for whether the biography describes the education entry as current or ongoing.

Position records represent professional roles mentioned in the biography. Each record keeps the `role`, `department`, `institution`, `location`, `start_year`, `end_year`, and `description` together when these values are stated. The `is_current` field indicates whether the biography describes the position as current or ongoing.

The schema uses simple JSON datatypes. Textual fields such as `full_name`, `degree`, `university`, `role`, and `institution` are represented as strings or `null`. Year fields such as `birth_year`, `edu_year`, `start_year`, and `end_year` are represented as integers or `null`. The `is_current` field is represented as a Boolean value. List-based fields, such as `research_areas`, `awards`, `memberships`, `activities`, `projects`, and `scholarly_metrics`, can contain multiple values.

The schema was selected based on the types of information commonly found in IEEE and ORCID biographies and on the requirements of the evaluation. By separating

author, education, and position information, the schema avoids merging several career facts into one ambiguous text field and allows education and position records to be evaluated separately.

The schema also defines how missing information is handled. If the biography text does not provide a value for a field, the model should use `null` for single-value fields or an empty list for list-based fields. For example, if a biography mentions a degree but does not include the graduation year, the `edu_year` field should be set to `null`. This rule is important because the goal is to extract information from the biography text only, not to infer or complete missing details from external sources.

The nested JSON output is later flattened into three CSV tables: an author table, an education table, and a position table. This structure allows repeated education and position records to be evaluated separately while keeping all records linked to the corresponding `dblp_pid`. The complete extraction prompt and schema are provided in Appendix A.

4.3 Prompt Design

The extraction pipeline is built on prompt-based extraction using large language models rather than fine-tuning a model for this specific task. The prompt connects the biography text to the extraction schema. It tells the model what to look for, how to structure the output, and which constraints to respect. This approach suited the thesis well because the output had to be machine-readable, conformant with a defined schema, and regular enough to validate and evaluate without manual intervention.

The prompt itself follows a schema-driven design. Rather than allowing the model to summarize the biography freely or pick out whatever it finds relevant, the prompt provides a fixed JSON structure and asks it to fill it using only what the biography actually says. Keeping the model anchored to a predefined format this way made the responses far more consistent across records and helped reduce issues such as field names that drift between responses, nesting that shifts depending on how a biography is written, or sentences of explanation appearing where a clean value was expected.

An important design choice is to separate the stable extraction instructions from the specific biography input. The prompt includes the extraction role, field definitions, JSON schema, formatting rules, and rules against unsupported inference. The biography text is provided as the unique input for each record. This way, the extraction instructions stay the same for all records, and only the biography content changes.

The prompt was refined during development based on preliminary model outputs. Early versions used more general instructions to extract the researcher's career information and return it as JSON. These versions showed that the model could produce inconsistent field names, merge related facts into one field, or include explanatory text around the JSON output. Later versions, therefore, introduced stricter output rules, clearer field descriptions, explicit null handling, and more detailed instructions for separating related fields. This refinement is consistent with recent work showing that prompt design affects knowledge extraction from text and that different prompt templates can lead to different extraction behaviour [PTG25].

The final prompt has three main parts. First, it explains that the task is to extract information from the researcher's biography. Second, it lists the extraction rules, such as using only information found in the biography and leaving missing values blank. Third, it gives the target JSON schema described in Section 4.2. The model should return only valid JSON, with no markdown, explanations, or extra comments.

A particular focus of the prompt design is the separation of related but distinct fields. In academic biographies, department, institution, field of study, location, and research area may appear close together in the same sentence. The prompt, therefore, defines these fields explicitly and instructs the model to keep related values grouped inside the correct education or position record. For example, a degree should remain associated with its university and year when these values are stated, while a professional role should remain associated with its department, institution, location, and time period.

The prompt also explains how to handle missing information. If the biography text does not support a field, the model should use `null` for single-value fields or an empty list for list-based fields. This rule matters because the goal is not to fill in missing biographical details from outside sources, but to turn the information given in the biography into a structured format.

The same schema and extraction logic are used for both IEEE and ORCID biographies. The datasets are processed in separate runs, but the expected output structure remains the same. The complete extraction prompt, including the full JSON schema, is provided in Appendix A.

4.4 Model and Deployment Choice

The extraction pipeline uses LLaMA 3.1 SauerkrautLM 70B Instruct as the production model. The model was selected because the thesis requires an instruction-following model that can generate structured JSON output and can be deployed in a controlled, self-hosted environment. This is important because the final extraction-ready dataset contains 416,679 biography records and must be processed in a repeatable batch workflow.

During development, several instruction models available in the interactive testing environment were explored using sample biography records and early versions of the extraction prompt. These tests were used to check whether the prompt could produce usable schema-conformant JSON outputs. They were not designed as a formal model benchmark, and the set of available models changed over time. Therefore, the thesis does not claim that the selected model is generally superior to other large language models.

LLaMA 3.1 SauerkrautLM 70B Instruct was selected for the production extraction because it produced usable schema-conformant outputs during preliminary prompt testing and was available in the self-hosted inference environment [VAG24]. The model choice is therefore treated as part of the pipeline design rather than as a separate model-comparison contribution.

The use of a self-hosted open-weight model also supports the practical requirements of the thesis. It avoids dependence on external commercial APIs, allows generation settings to be controlled, and supports long-running batch extraction over large input

files. The concrete implementation of the inference setup, including the use of vLLM and SLURM, is described in Chapter 5.

The quality of the selected extraction setup is assessed through the field-level evaluation presented in Chapter 6.

4.5 Pipeline Architecture

The extraction pipeline follows a modular workflow that takes prepared biography records and turns them into structured outputs ready for evaluation. The main stages are input preparation, LLM inference, output validation, and tabular export. Keeping these stages separate makes the pipeline easier to follow and means each one can be checked and tested independently without touching the others.

The workflow begins with input preparation. The cleaned biography records described in Chapter 3 are converted into the input format expected by the extraction script. Each record contains the selected biography text and the identifiers needed to connect the generated output back to the original record. The `dblp_pid` is kept as the researcher-level identifier, while `doi` or `orcid` is retained depending on the source.

After input preparation, each biography is processed by the language model. The biography text is combined with the extraction prompt described in Section 4.3, and the model generates a nested JSON object following the schema described in Section 4.2. IEEE and ORCID biographies are processed in separate runs, but the same schema and prompt logic are used for both sources.

Once the model has produced an output, it goes through a validation step before anything else happens with it. Language model responses do not always come back in exactly the expected format, so this step checks whether the output can be parsed as valid JSON, whether the required fields are present, and whether missing values have been represented correctly. Any output that does not pass these checks is handled separately and flagged accordingly, rather than being quietly included in the final results.

After validation, the nested JSON structure is flattened into three CSV tables covering authors, education, and positions. The `dblp_pid` is carried across all three tables so that every row can be traced back to the biography it came from. This format makes the extracted data straightforward to compare against the gold standard at the level of individual fields.

Figure 4.1 illustrates the high-level architecture of the pipeline.

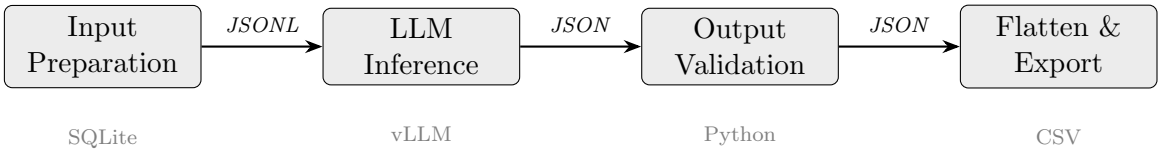


Figure 4.1: High-level architecture of the extraction pipeline.

This architecture follows the requirements derived in Chapter 3. It preserves researcher identifiers, applies a unified schema to both biography sources, supports nested education and position records, and produces CSV outputs that can be evaluated systematically.

4.6 Summary

This chapter presented the design and conception of the extraction pipeline. The design is guided by the requirements derived from the dataset analysis in Chapter 3. The main goals are source-independent extraction, schema consistency, support for one-to-many career structures, controlled extraction without unsupported inference, scalability, and evaluation readiness.

The extraction schema was organized into three output groups: author records, education records, and position records. This structure allows researcher-level attributes, education information, and professional positions to be represented consistently. The nested JSON format preserves repeated education and position records and can later be flattened into an author table, an education table, and a position table for evaluation.

The prompt design follows a schema-driven strategy. The model is instructed to extract only information stated in the biography text, follow a fixed JSON structure, handle missing values explicitly, and avoid explanatory text. The prompt was refined during development to improve structural consistency, reduce unsupported inference, and clarify field boundaries.

The chapter also explained the model and deployment choice. LLaMA 3.1 SauerkrautLM 70B Instruct is used as the production model in a self-hosted environment, while the quality of the selected extraction setup is assessed through the evaluation in Chapter 6. Finally, the pipeline architecture was described as a modular workflow consisting of input preparation, LLM inference, output validation, and tabular export. The following chapter explains how this design was implemented in practice.

5. Implementation

This chapter describes how the extraction pipeline from Chapter 4 was implemented. The process begins with the cleaned IEEE and ORCID biography records and results in structured CSV files for evaluation. The main areas of focus include input preparation, batch execution, LLM inference, output validation, and export.

5.1 Input Data Preparation and Batch Organization

The implementation starts with the cleaned IEEE and ORCID biography records prepared in Chapter 3. These records are formatted uniformly so that the same extraction script can handle both sources. Each record contains the biography text and identifiers that link the generated output back to the source data. The key identifier is `dblp_pid`; `doi` is kept for IEEE records and `orcid` for ORCID records.

Although the records have unique identifiers, the extraction process relies on the biography text. The identifiers are not used to support extraction; they exist to maintain the link between the input record, the raw model response, the validated JSON output, and the final CSV rows. This differentiation is important since the focus is on extracting information from the biography text rather than populating records with external metadata.

The records are exported in JSONL format, with each line representing one biography record. This line-oriented format is ideal for large-scale processing because it allows records to be read independently and divided into smaller processing units. Listing 5.1 shows a simplified anonymized input record, formatted over multiple lines for clarity.

Listing 5.1: Simplified anonymized JSONL input record.

```
{
  "dblp_pid": "example/researcher-001",
  "doi": "10.xxxx/example.doi",
  "biography": "Researcher A received the Ph.D. degree from
     University X in 2018. She is currently an Associate
     Professor at University Y. Her research interests include
      information extraction and knowledge graphs."
}
```

ORCID records use the same structure as IEEE records, with one small difference: instead of a `doi`, each record uses an `orcid` as its identifier. Because of this, both sources can use the same extraction workflow with only minor adjustments. The identifier is simply stored under a different field name, which makes it easy to track during evaluation.

During implementation, SQLite was used as a lightweight way to manage intermediate records during intermediate processing. It stores each record alongside its identifier and tracks whether it has already been processed. This made it easy to inspect records before sending them to the model, avoid reprocessing records that were already done, and link each output back to the input it came from. SQLite is not the final storage format. It is just a practical tool for staying organized while the data moves through the pipeline.

Managing the records in a database also made it easier to handle the scale of the dataset. With over 416,000 biography records in total, trying to process everything as a single file would be difficult to monitor and nearly impossible to recover from if an error occurred. Instead, the records were split into smaller chunks, each processed as a separate job with its own output and log file. If a job failed or was interrupted, only that chunk needed to be rerun and not the entire dataset.

Before running the full process, smaller test samples from both IEEE and ORCID records were used to verify that the pipeline worked from input preparation to output export. These samples helped verify that the input format was correct, the prompt was configured properly, the model responded as expected, the JSON output could be parsed, and the final CSV export produced the expected results. This step was important because a small bug in one record can easily cause problems for thousands of records during large-scale processing.

5.2 Batch Processing and HPC-based LLM Inference

The biography dataset prepared for extraction contained 416,679 records. Running all of them through the model in a single run was simply not feasible, so the workload was broken into smaller chunks that could each be processed on their own. Dividing the workload in this way made it much easier to keep track of progress and meant that if an error occurred, only the affected chunk needed to be rerun rather than the entire dataset.

Scheduling across the cluster was handled using SLURM job arrays. Each task in the array took responsibility for a specific slice of the input and saved its results to dedicated output and log files. This kept jobs independent from one another, so a timeout, a node failure, or any other cluster-side issue would affect only one task and leave everything else intact. The pipeline was also built to resume from the last completed record. Each time the script started processing a chunk, it first checked which records had already produced output and skipped them. This turned out to be important in practice because long-running jobs on a shared cluster do not always finish cleanly. With resume logic in place, a restarted job could resume from the last completed record rather than repeating work that had already been done.

When processing each record, the script combined the extraction prompt with the biography text to form the model input. The record identifiers `dblp_pid`, `doi`, and `orcid` are carried alongside the data throughout the pipeline for traceability purposes, but they are never passed to the model itself. The model only receives the biography text and is expected to extract structured information from that text alone.

Inference was carried out using LLaMA 3.1 SauerkrautLM 70B Instruct [VAG24], served through vLLM [KLZ+23] on the KISSKI/GWDG HPC environment [GWD26].

vLLM was selected primarily for how it handles memory, using a technique called PagedAttention that makes serving large models considerably more efficient. Without access to the HPC environment, running a 70-billion-parameter model across this volume of records would not have been realistic.

The temperature is set to zero for all inference runs to keep the model's behavior consistent and predictable. This was important because the outputs had to be compared with manually annotated gold standard records. The raw model response was always saved to disk before validation or post-processing. Some outputs contained formatting issues or missing fields, so these cases were handled separately instead of being discarded.

The IEEE and ORCID datasets were each processed in separate extraction runs. The runtimes shown in Table 5.1 are approximate wall-clock estimates, not results from controlled throughput tests. Because the runs took place on a shared HPC system, completion times could change based on queue wait times, scheduler choices, available resources, batch setup, and differences in biography lengths. They also do not scale linearly with dataset size because the workload was processed in parallel using SLURM job arrays. ORCID has fewer records than IEEE, but its biographies are generally longer and more varied. For these reasons, the reported runtimes should be understood as practical estimates for the full extraction runs rather than as direct per-record speed comparisons.

Table 5.1: Approximate wall-clock runtime for full dataset extraction.

Dataset	**Extraction-ready records**	**Runtime**
IEEE	390,920	36 hours
ORCID	25,759	18 hours
Total	**416,679**	**54 hours**

Listing 5.2 illustrates the batch assignment logic in simplified form. The variable `chunk_size` defines how many input records are assigned to one batch job. The variable `task_id` represents the SLURM array task number. This task number is used to calculate the start and end positions of the records processed by that job. For example, with a chunk size of 1,000 records, task 0 processes records 0–999, task 1 processes records 1,000–1,999, and task 2 processes records 2,000–2,999.

Listing 5.2: Simplified batch assignment logic.

```
chunk_size = 1000

Example assignment:

task_id = 0  -> records 0 to 999

task_id = 1  -> records 1000 to 1999

task_id = 2  -> records 2000 to 2999

task_id = 3
```

```
start_index = task_id * chunk_size
end_index = start_index + chunk_size

input_records = records[start_index:end_index]
output_file = "outputs/batch_003.jsonl"
```

This batch-oriented implementation made the extraction process scalable and recoverable. Instead of relying on one monolithic inference run, the dataset was processed through independent chunks with separate logs, output files, and resume checks.

5.3 Output Validation, Error Handling, and CSV Export

Once inference is complete, the results are stored as a JSONL file on the HPC system. Each line in that file represents one processed record and contains the source identifier, the `dblp_pid`, the extracted entities, and a metadata block. The metadata captures processing details such as the input line number, model path, prompt file, timestamp, any warnings that were raised, and whether the record produced an error. These fields are not part of the extraction output itself; they exist to support traceability and to make it straightforward to separate clean outputs from those that need attention.

Listing 5.3 shows a simplified anonymized JSONL output record produced by the extraction pipeline. The example corresponds to the simplified input record shown earlier, but only selected fields are shown for readability.

Listing 5.3: Simplified anonymized JSONL extraction output record.

```
{
  "dblp_pid": "example/researcher-001",
  "doi": "10.xxxx/example.doi",
  "entities": {
    "author": {
      "full_name": "Researcher A",
      "birth_year": null,
      "birth_location": null,
      "nationality": null,
      "research_areas": [
        "information extraction",
        "knowledge graphs"
      ],
      "awards": [],
      "memberships": [],
      "activities": [],
      "projects": [],
      "scholarly_metrics": [],
      "raw_facts": []
    },
    "education": [
      {
        "degree": "Ph.D.",
        "field": null,
        "university": "University X",
```

```
        "location": null,
        "edu_year": 2018,
        "is_current": false
      }
    ],
    "positions": [
      {
        "role": "Associate Professor",
        "department": null,
        "institution": "University Y",
        "location": null,
        "start_year": null,
        "end_year": null,
        "is_current": true,
        "description": null
      }
    ]
  },
  "metadata": {
    "has_error": false,
    "warnings": [],
    "needs_review": false
  }
}
```


The output record preserves the identifiers from the input and stores the extracted information under the three schema groups: author, education, and positions. Fields that are not explicitly stated in the biography are represented as `null` or empty lists. The metadata block is used for implementation-level tracking, such as parse errors, warnings, and records that may require later inspection.

Even though the prompt explicitly asks the model to return only JSON, the implementation does not assume that every response will be well-formed. A model response can include formatting markers, escaped newline characters, trailing commas, extra text, or syntax that breaks JSON parsing. To handle this, each raw response is cleaned before parsing is attempted. Surrounding formatting is stripped where possible, and minor issues are corrected automatically. If the response still cannot be parsed after these corrections, the record is written to the output file with an error label of `parse_failed` and the `has_error` flag is set in the metadata.

Errors that happen before or during inference are handled in a similar way. If an input record does not contain a usable biography, it is marked with the label `no_biography`. If an entire inference batch fails, the affected records receive the label `batch_exception`. None of these cases is counted as a successful extraction, but they are all preserved in the JSONL output so that nothing is silently lost. The error label and metadata fields, such as `has_error`, `warnings`, and `needs_review`, make them easy to identify and investigate later.

Once a response is parsed, its structure is checked to make sure it matches the expected schema. Author-level fields, education entries, and position entries are each validated on their own. Single-value fields should have either a value or `null`, while list-based fields should contain lists. Education and position entries need to follow the

nested structure described in Section 4.2. Each entry should use dedicated fields like `degree`, `university`, `role`, `institution`, and `edu_year`, rather than combining all information into one string.

Where the structure is close but not quite correct, minor inconsistencies are corrected automatically. For example, if a field such as `research_areas`, `awards`, `projects`, `memberships`, `activities`, or `raw_facts` is returned as a plain string instead of a list, it is wrapped into a list. If a position entry is missing the `is_current` field, the implementation infers a Boolean value from whether `end_year` is present. Importantly, none of these corrections are applied silently; each correction is recorded as a warning in the metadata so that the change remains visible and traceable.

Once validation is complete, the JSONL extraction output is converted into CSV files. This process separates information about each biography from repeated education and position entries. The author table has one row for each biography, while the education and position tables can have several rows for the same `dblp_pid`. This setup keeps one-to-many career details clear, without combining repeated degrees or positions into a single field. It also makes it easier to evaluate each field, since author details, education records, and position records can be checked separately against the manually annotated gold standard data.

Alongside these CSV files, the evaluation workflow can also produce spreadsheet files for manual review. These `.xlsx` files are not the main output of the pipeline; they are inspection aids that make it easier to review field-level comparisons, spot mismatches, and work through evaluation details by hand. The primary structured outputs of the pipeline remain the three CSV tables, while the spreadsheet files serve the evaluation analysis described in Chapter 6. The resulting CSV files form the direct input for the evaluation workflow described in Chapter 6.

5.4 Summary

This chapter described how the extraction pipeline works. The cleaned IEEE and ORCID biography records were changed into a uniform JSONL input format while keeping the `dblp_pid` and source-specific identifiers intact. SQLite served as an intermediate record-management tool to support inspection, traceability, and batch organization.

The extraction was set up as a batch workflow because the final dataset includes 416,679 biography records. Input records were divided into smaller groups and processed in the HPC environment using SLURM job execution. Resume logic allowed interrupted jobs to continue without repeating completed records. The complete extraction process took about two days for the IEEE dataset and one day for the ORCID dataset.

LLM inference used LLaMA 3.1 SauerkrautLM 70B Instruct served through vLLM in the self-hosted KISSKI/GWDG computing environment. The model operated with deterministic generation settings and the schema-driven prompt detailed in Chapter 4.

After inference, the JSONL extraction outputs were checked for parsing errors, warnings, and schema-related issues. The output records contain metadata such as `has_error`, `warnings`, and `needs_review`, which help differentiate problematic

records during later processing. The extracted data was then organized into three CSV tables: an author table, an education table, and a position table. These CSV files provide the main structured output for evaluation, while additional spreadsheet files are created in the evaluation workflow to assist with manual review and verification.

The next chapter explains the evaluation setup, gold standard construction, metrics, and extraction results.

6. Evaluation and Results

This chapter evaluates the schema-driven extraction pipeline discussed in earlier chapters. It focuses on how accurately the pipeline extracts structured researcher career details from IEEE and ORCID biography texts. The comparison takes place at the field level by aligning the LLM output with manually annotated gold standard data. First, the chapter describes the gold standard and evaluation files, then it explains the metrics, matching, and normalization strategy. Lastly, it presents the extraction results and conducts an error analysis with examples.

6.1 Gold Standard and Evaluation Files

The evaluation depends on a manually annotated gold standard. A total of 150 biography records were annotated: 100 from the IEEE dataset and 50 from the ORCID dataset. The gold standard matches the structure of the extraction pipeline. The gold standard matches the structure of the extraction pipeline and is organized into authors, education, and positions. The author group includes researcher-level fields such as `full_name`, `birth_year`, `birth_location`, `nationality`, `research_areas`, `awards`, `memberships`, `activities`, `projects`, `scholarly_metrics`, and `raw_facts`, which is evaluated as part of the author-level output rather than as a separate entity group. This method enables a comparison between each field in the LLM output and the manual annotations.

The evaluation is limited to English biography texts. This scope was chosen because the prompt, annotation process, and comparison rules were designed specifically for English-language biographies. This is especially relevant for ORCID, where researcher-maintained profiles might include non-English biography text. Such cases fall outside the current evaluation and will be addressed as a limitation in Chapter 7.

During annotation, only the information clearly stated in the biography was included. No additional information about researchers from outside sources or institutions was included. For example, if a biography stated that a researcher earned a Ph.D. from University X but did not mention the graduation year, the degree and university were recorded, while the `edu_year` field was left empty. This rule ensures that the evaluation uses only information found in the biography and does not add details from other sources.

The following simplified example illustrates how a biography text was converted into the manual gold standard. The example is anonymized and does not represent a real researcher's record. It only shows how explicitly stated information is mapped to the author, education, and position output groups.

> Dr. A received the Ph.D. degree in Computer Science from University X in 2018. She is currently an Associate Professor in the Department of Computer Science at University Y in City Z. Her research interests include machine learning and data mining.

Table 6.1: Simplified example of a manual gold standard annotation.

Output group	**Field or entry**	**Manual gold standard value**
Author	`full_name`	Dr. A
Author	`research_areas`	machine learning; data mining
Education	Entry 1	`degree = Ph.D.; field = Computer Science; university = University X; edu_year = 2018`
Position	Entry 1	`role = Associate Professor; department = Department of Computer Science; institution = University Y; location = City Z; is_current = true`

This example shows that the gold standard records only the information stated in the biography. Values that are not stated, such as a birth year, nationality, start year, or end year in this example, remain empty in the manual annotation.

The manual annotations were stored in separate files for the three output groups. For IEEE, the gold standard files are `ieee_manual_authors.csv`, `ieee_manual_education.csv`, and `ieee_manual_positions.csv`. For ORCID, the gold standard files are `orcid_manual_authors.csv`, `orcid_manual_education.csv`, and `orcid_manual_positions.csv`.

The LLM output was flattened into the same three groups. For IEEE, the corresponding LLM output files are `ieee_authors_llm.csv`, `ieee_education_llm.csv`, and `ieee_positions_llm.csv`. For ORCID, the corresponding files are `orcid_authors_llm.csv`, `orcid_education_llm.csv`, and `orcid_positions_llm.csv`. This structure allows the evaluation script to compare each manual file with the corresponding LLM output file. For IEEE, the comparison uses `doi` and `dblp_pid`; for ORCID, it uses `orcid` and `dblp_pid`.

The evaluation workflow contains four main file groups.

Table 6.2: Main files used in the evaluation workflow.

File type	Purpose in the evaluation
Manual gold standard CSV files	Human-created reference annotations for author, education, and position records. These files are used as the ground truth for evaluation.
LLM CSV files	Flattened LLM outputs for author, education, and position records. These files are compared with the corresponding manual gold-standard files.
Metrics CSV file	Aggregated field-level counts and scores, including TP, TN, FP, FN, accuracy, precision, recall, and F1-score.
Review XLSX files	Inspection files containing manual vs. LLM comparisons and count sheets for authors, education, and positions.

The review files help make it easier to check the evaluation results. Each file has a `manual_vs_llm` sheet and a `count_sheet`. The `manual_vs_llm` sheet shows the manual value, the LLM value, the field-level outcome, and the similarity score if there is one. The `count_sheet` sums up the outcomes as TP, TN, FP, and FN counts for each field. These files support manual checking of the evaluation results, while the main result tables in this chapter show the most important overall scores.

Here is an anonymized example that shows how an education entry is compared to the manually annotated gold standard. This example does not come from a real researcher's record. It is meant to show the evaluation structure for specific education fields.

Table 6.3: Simplified example of manual gold standard and LLM output comparison for an education record.

Field	Manual gold standard	LLM output
`dblp_pid`	researcher_001	researcher_001
`full_name`	Dr. A	Dr. A
`degree`	Ph.D.	Ph.D.
`university`	University X	University X
`field`	Computer Science	Computer Science
`edu_year`	2018	2018
`location`	empty	City Y
`is_current`	false	false

In this example, `degree`, `university`, `field`, `edu_year`, and `is_current` would be counted as correct matches. The `location` field would be counted as a false positive since the LLM provided a location that the manual gold standard did not include. This kind of field-level comparison serves as the basis for the evaluation process explained in the next section.

6.2 Metrics, Matching, and Normalization

The evaluation happens at the field level. Each field in the manual gold standard is compared with the matching field in the LLM output. The evaluation does not

just check if a complete biography record is correct or incorrect. Instead, it looks at author fields, education fields, and position fields one by one. This method helps identify which parts of the schema are extracted consistently and which fields remain difficult.

The evaluation script assigns one of four outcome labels to each field comparison: true positive, true negative, false positive, or false negative. These labels are aggregated into the result tables in Section 6.3. Understanding the labels is easier with an example. Table 6.4 shows the possible outcomes for the field `location`.

Table 6.4: Example of TP, TN, FP, and FN outcomes for the field `location`.

Outcome	Manual gold standard	LLM output	Explanation
TP	Qatar	Qatar	The location is stated in the gold standard and the model extracts the same value.
TN	empty	empty	No location is stated in the gold standard and the model also leaves the field empty.
FP	empty	Qatar	The model predicts a location although the gold standard does not contain one.
FN	Qatar	empty	The gold standard contains a location, but the model does not extract it.
FN	Qatar	Trier	The gold standard contains `Qatar`, but the model extracts a different location. Under strict matching, the correct gold value is therefore missed.

This focus on individual fields is important because a single biography can contain both correct and incorrect extractions. For example, the model may correctly extract a degree and university but include a location not found in the biography. In this case, the degree and university are seen as correct, while the location is seen as a mistake. The whole record is not simply labeled as right or wrong.

Different field types are compared in various ways. Numeric fields like `birth_year`, `edu_year`, `start_year`, and `end_year` are treated as numbers. Boolean fields such as `is_current` are changed to true or false values. Text fields require more processing because the same information can be expressed in different ways.

For text fields, the evaluation compares the generated strings and field values. It does not perform entity linking. This means it does not connect an institution, department, or location to an external identifier. Instead, it verifies if the string produced by the model is sufficiently similar to the string in the gold standard after cleaning and normalization. This distinction is crucial because two strings can describe the same entity but still be challenging to compare automatically.

Table 6.5 includes examples of string variations and field-boundary issues that can affect the evaluation.

Table 6.5: Examples of string comparison and normalization issues.

Manual gold standard	LLM output	Evaluation issue
Department of Computer Science	Computer Science Department	The same department is expressed with a different word order.
University X	Dept. of Computer Science, University X	The model includes department information inside the institution field.
Computer Science Department	University X	The model extracts the institution but misses the department.
Trier, Germany	Germany, Trier	The location contains the same parts but in a different order.

These examples show why normalization is needed before comparing text fields. Minor differences in capitalization, punctuation, word order, or abbreviations should not automatically be treated as extraction errors. However, normalization also has limits. If the model places a department inside the institution field, or extracts only the institution while missing the department, this is not only a spelling variation but a field-boundary error.

Location fields require an additional normalization step because the same place can be written in different orders. For example, `Trier, Germany` and `Germany, Trier` contain the same two location parts but in reversed order. In such cases, the evaluation treats the two comma-separated parts as comparable after normalization. This helps avoid counting simple order differences as extraction errors. However, this is still a string-based comparison. The evaluation does not geocode the location, does not resolve it to a geographic identifier, and does not check whether two different place names refer to the same real-world location.

The evaluation therefore works with normalized strings, not with fully resolved entities. For example, `Department of Computer Science` and `Computer Science Department` may refer to the same department, but the evaluation does not confirm this by linking both strings to an external department entity. It only compares the text values using the implemented cleaning and similarity rules. As a result, some semantically close values may still be counted as mismatches if their string forms or field boundaries differ too much.

Before field-level comparison, the manual and LLM outputs must also be matched at the row level. Author records are matched using the source-specific identifiers. For IEEE, the matching uses `doi` and `dblp_pid`; for ORCID, it uses `orcid` and `dblp_pid`. Education and position records require additional handling because one researcher may have multiple education entries or multiple positions.

For education records, one researcher may have more than one degree. The evaluation therefore first aligns the manual and LLM education rows belonging to the same researcher before comparing the individual fields. After this row-level alignment, fields such as `degree`, `field`, `university`, `location`, `edu_year`, and `is_current` are evaluated separately. This avoids comparing all education information as one

combined text block and allows each education entry to be evaluated as a separate structured record.

For position records, matching is more difficult because biographies may mention several roles in different orders. The evaluation pairs manual and LLM position entries using role and institution similarity before comparing the individual fields. This reduces the risk that a correct position is counted as wrong only because it appears in a different order in the LLM output. After the position entries are paired, fields such as `role`, `department`, `description`, `institution`, `location`, `start_year`, `end_year`, and `is_current` are evaluated.

The evaluation also includes existence checks, such as `education_exists` and `position_exists`. These fields check whether the model detected that at least one education or position entry is present in the biography. For example, if the gold standard contains at least one position and the LLM output also contains at least one position, `position_exists` is counted as a true positive. If neither contains a position, it is counted as a true negative.

The review files preserve these comparisons in an inspectable format. For each evaluated field, the review sheet stores the manual value, the LLM value, the outcome label, and the similarity score where applicable. The count sheet then aggregates these outcomes into field-level counts and scores. These aggregated values are used for the result tables in the following section.

6.3 Extraction Results

This section presents the extraction results for IEEE and ORCID biographies. The results are reported at two levels. First, Table 6.6 summarizes the micro-averaged scores for the three output groups: authors, education, and positions. Second, Tables 6.7 to 6.9 compare the field-level results for IEEE and ORCID side by side. This makes it possible to identify which fields are extracted reliably and where the two sources differ.

Table 6.6: Micro-averaged extraction results by source and entity group.

Source	Entity group	Accuracy	Precision	Recall	F1
IEEE	Authors	0.980	0.974	0.963	0.969
IEEE	Education	0.990	0.998	0.991	0.995
IEEE	Positions	0.940	0.945	0.953	0.949
ORCID	Authors	0.940	0.963	0.854	0.905
ORCID	Education	0.947	0.986	0.946	0.966
ORCID	Positions	0.897	0.942	0.886	0.913

The micro-averaged results show strong overall extraction performance for both sources. IEEE scores higher than ORCID across all three entity groups. This reflects the more standardized style of IEEE biographies compared to the more varied researcher-maintained ORCID biographies. The field-level tables below provide more detail on these differences.

The following field-level tables report TP, TN, FP, FN, and F1-score for each evaluated field. These counts refer to field-level comparisons, not the number of researchers.

Accuracy, precision, and recall are part of the micro-averaged summary above, while the field-level tables focus on counts and F1-score to keep the comparison streamlined.

Table 6.7: Author entity field comparison in IEEE and ORCID.

Field	IEEE					ORCID				
	TP	TN	FP	FN	F1	TP	TN	FP	FN	F1
activities	2	94	1	3	0.500	9	35	0	6	0.750
awards	9	91	0	0	1.000	10	39	0	1	0.952
birth_location	36	64	0	0	1.000	2	48	0	0	1.000
birth_year	34	66	0	0	1.000	3	47	0	0	1.000
full_name	100	0	0	0	1.000	36	14	0	0	1.000
memberships	5	94	0	1	0.909	11	34	0	5	0.815
nationality	35	64	0	1	0.986	3	47	0	0	1.000
projects	3	93	3	1	0.600	8	41	1	0	0.941
raw_facts	5	88	5	2	0.588	21	16	3	10	0.764
research_areas	95	2	0	3	0.985	36	9	2	3	0.935
scholarly_metrics	16	82	0	2	0.941	19	29	0	2	0.950
Micro-average	**340**	**738**	**9**	**13**	**0.969**	**158**	**359**	**6**	**27**	**0.905**

Table 6.8: Education entity field comparison in IEEE and ORCID.

Field	IEEE					ORCID				
	TP	TN	FP	FN	F1	TP	TN	FP	FN	F1
degree	190	1	0	5	0.987	67	1	1	5	0.957
edu_year	136	58	0	2	0.993	36	34	0	4	0.947
education_exists	100	0	0	0	1.000	34	16	0	0	1.000
field	131	61	2	2	0.985	58	10	2	4	0.951
full_name	196	0	0	0	1.000	66	8	0	0	1.000
is_current	196	0	0	0	1.000	74	0	0	0	1.000
location	127	68	0	1	0.996	30	39	2	3	0.923
university	194	0	0	2	0.995	56	9	1	8	0.926
Micro-average	**1270**	**188**	**2**	**12**	**0.995**	**421**	**117**	**6**	**24**	**0.966**

Table 6.9: Position entity field comparison in IEEE and ORCID.

Field	IEEE					ORCID				
	TP	TN	FP	FN	F1	TP	TN	FP	FN	F1
department	43	67	2	11	0.869	53	44	3	22	0.809
description	4	110	8	1	0.471	4	105	11	2	0.381
end_year	17	98	7	1	0.810	26	93	1	2	0.946
full_name	123	0	0	0	1.000	110	11	0	1	0.996
institution	109	4	5	5	0.956	109	0	3	10	0.944
is_current	118	0	1	4	0.979	109	0	4	9	0.944
location	50	58	13	2	0.870	33	71	10	8	0.786
position_exists	64	34	2	0	0.985	40	10	0	0	1.000
role	102	14	0	7	0.967	97	1	2	22	0.890
start_year	43	77	1	2	0.966	38	76	4	4	0.905
Micro-average	**673**	**462**	**39**	**33**	**0.949**	**619**	**411**	**38**	**80**	**0.913**

The field-level results indicate that education fields are consistently strong across both sources. Author fields are also reliable for well-defined attributes, especially `full_name`, `birth_year`, `birth_location`, and `nationality`. Lower scores mainly occur in open-ended fields such as `activities`, `projects`, `raw_facts`, and `description`, where the boundaries for extraction are less clear. Position extraction faces additional challenges in fields such as `department`, `location`, and `role`, as these values often appear close to institution names or other career details within the same sentence.

6.4 Error Analysis with Examples

This section analyses the main error patterns observed in the evaluation. The analysis is based on the same field-level comparison outputs used for the extraction results in Section 6.3. Errors are counted from false positive and false negative field-level decisions. A false positive means that the model produced a value that was not present in the manual gold standard, while a false negative means that the manual gold standard contained a value that the model either missed or did not match under the evaluation rules.

In addition to the field-level scores, the error-analysis script groups selected false positive and false negative cases into diagnostic categories. These categories summarize recurring patterns observed in the comparison output, such as institution-name variation, department–institution boundary problems, location handling errors, open-ended field errors, and multi-entry merging. Table 6.10 reports selected diagnostic error categories generated during error inspection. It is not an exhaustive table of all field-level errors; remaining mismatches that do not fall into these categories are not shown in the compact table. In addition, multi-entry merging is counted from unmatched aligned education or position entries. Therefore, the values in this table are used for diagnostic interpretation and are not intended to sum to the overall FP and FN totals in the field-level result tables.

Table 6.10: Selected diagnostic error categories in IEEE and ORCID. Counts refer to selected FP and FN observations after row alignment; multi-entry merging is counted from unmatched education or position entries.

Error type	**IEEE**			**ORCID**		
	FP	**FN**	**Total**	**FP**	**FN**	**Total**
Institution name mismatch	5	7	12	4	10	14
Department boundary ambiguity	2	11	13	3	22	25
Location hallucination	13	0	13	12	0	12
Location omission	0	1	1	0	3	3
Multi-entry merging	3	37	40	15	32	47
Open-ended field error	17	7	24	15	18	33
Nationality confusion	0	1	1	0	0	0

Within the selected diagnostic categories shown in Table 6.10, multi-entry merging has the highest total count in both IEEE and ORCID. This indicates that the model sometimes combines several education or position entries into fewer structured

records. Open-ended field errors are also frequent in both sources, especially because fields such as `activities`, `projects`, `raw_facts`, and `description` can overlap in meaning. Department boundary ambiguity is another relevant error type, particularly for ORCID.

Institution name mismatch also remains relevant. This occurs when the manual annotation and the model output contain different surface forms of the same or similar institution name. For example, the gold standard may contain only the main university name, while the model output includes a department, school, or research center together with the institution name.

A related qualitative observation was made during manual inspection of institution-name mismatches. In some cases, the model extracted the main institution name but omitted information written in parentheses. This parenthetical information may contain an acronym, organizational unit, campus label, or alternative institution name. Since this pattern was not counted as a separate diagnostic category, it is not reported separately in Table 6.10. Instead, it is treated as part of the broader institution-name mismatch problem.

Department boundary ambiguity occurs when the model does not clearly separate the department from the institution. Academic biographies often express affiliations as long chains, such as department, laboratory, faculty, university, and location. The model may place too much of this chain into one field or assign a department value to the institution field.

In the selected diagnostic categories, location errors are reported in two forms. A location hallucination occurs when the model adds a location that is not explicitly stated in the biography. A location omission occurs when the biography states a location, but the model leaves the location field empty. This is important because the prompt explicitly instructs the model not to infer locations from institution names or world knowledge. Other location-related mismatches may occur when both values are present but do not match under the evaluation rules; these cases are not shown as a separate row in the compact diagnostic table.

The following simplified examples demonstrate how the main error types appear in the manual versus model comparison. These examples are anonymized and do not represent real researcher records.

Table 6.11: Examples of recurring extraction errors.

Error type	Example and interpretation
Institution name mismatch	**Entity:** position. Manual: `institution = University X`. LLM: `institution = Department of Computer Science, University X`. The model includes department information inside the institution field.
Department boundary ambiguity	**Entity:** position. Manual: `department = Department of Computer Science`, `institution = University X`. LLM: `department = null`, `institution = Department of Computer Science, University X`. The model does not separate the department from the main institution.
Location hallucination	**Entity:** education. Manual: `location = null`. LLM: `location = City Y`. The model predicts a location although it is not stated in the biography.
Location omission	**Entity:** position. Manual: `location = City Y`. LLM: `location = null`. The biography states a location, but the model does not extract it.
Multi-entry merging	**Entity:** position. Manual: `Research Fellow, Institute X` and `Professor, University Y`. LLM: `Research Fellow and Professor, Institute X / University Y`. Two separate positions are merged into one output entry.
Open-ended field error	**Entity:** author. Manual: `activities = session chairing`. LLM: `raw_facts = session chairing`. The information is relevant, but it is assigned to a different open-ended field.

These examples show that many errors are not just caused by missing values. Often, they happen because of decisions about where one field ends and another begins. For example, `Department of Computer Science, University X` includes both a department and an institution. If the model puts the whole phrase in the institution field, it extracts useful information but assigns it to the wrong field.

The error analysis also shows why open-ended fields are harder to judge. Fields like `activities`, `projects`, `raw_facts`, and `description` often overlap in meaning. For example, a sentence about organizing a workshop, leading a program, chairing a session, or giving an invited talk can be interpreted in different ways depending on the schema rules. Because of this, these fields have more boundary errors than clear fields like `full_name`, `birth_year`, or `is_current`.

Overall, the main challenges come from unclear boundaries between institutions and departments, handling locations, merging multiple entries, and defining open-ended fields. These errors do not weaken the overall extraction results, but they show where more normalization, stricter schema rules, or post-processing are needed before using the data to improve a knowledge graph.

6.5 Summary

This chapter reviews the extraction pipeline by using a manually annotated gold standard of 150 biographies, including 100 from IEEE and 50 from ORCID. The

evaluation compared the manual gold standard with the model's output for each field, measuring true positives, true negatives, false positives, and false negatives. Instead of entity linking, the comparison used normalized string matching.

The results show strong extraction performance for both sources. IEEE biographies usually score higher than ORCID, likely because IEEE author biographies follow a more standardized format, while ORCID biographies vary more. Education extraction works best for both sources. Extracting position information is harder, since roles, departments, institutions, locations, and time periods often appear close together in the same sentence.

The field-level results show that clearly defined fields are extracted reliably across the different entity groups. For example, `full_name`, `birth_year`, and `birth_location` at the author level, and `is_current` in education and position records, are extracted well. In contrast, open-ended fields such as `activities`, `projects`, `raw_facts`, and `description` are harder to extract because their boundaries are less clear.

The error analysis shows that the main problems are mismatched institution names, unclear department boundaries, missing or wrong locations, and combining several entries into one record. These issues mostly come from deciding where field boundaries are, not from missing extractions.

Overall, the evaluation shows that the proposed extraction method can produce useful structured information from academic biographies, especially for well-defined schema fields. However, the results also show that more normalization, stricter schema rules, and post-processing are needed before the extracted data can be used reliably for knowledge graph enrichment.

7. Discussion, Conclusion, and Future Work

This chapter discusses the findings of the thesis and reflects on the extent to which the research objectives were addressed. It also summarizes the main limitations of the approach and outlines possible directions for future work.

7.1 Discussion of Findings

The aim of this thesis was to build and evaluate a scalable, schema-driven pipeline for turning academic biography text into structured researcher career information. To get there, the work involved designing a unified schema, developing a prompt-based extraction approach, running the pipeline across large IEEE and ORCID biography datasets, and checking the results against a manually annotated gold standard.

The results show that schema-driven prompt-based extraction can produce genuinely useful structured information from academic biographies. The pipeline was able to extract author-level attributes, education entries, and professional positions from biography texts that varied considerably in style and completeness. Using a unified schema made it possible to represent multiple education and position records for the same researcher without losing the thread back to the `dblp_pid` that identifies them.

The evaluation indicates that the approach works best when the information in the biography is clearly stated and follows a recognizable pattern. Education extraction came out strongest across both sources, which makes sense; degree type, university name, and graduation year tend to appear in fairly predictable phrases that the model can pick up on reliably. Author-level fields such as `full_name`, `birth_year`, `birth_location`, and `nationality` were also extracted well when the biography mentioned them directly.

Extracting position information was more difficult. Role, department, institution, location, and time period often show up together in the same sentence, so the model needs to do more than just find the right details. It also has to decide which part belongs to which field. The error analysis showed that the main problems were mismatched institution names, confusion about where department names start and end, inconsistent handling of locations, and situations where several position entries were combined into one.

A study published in March 2026 offers a useful point of comparison for these findings. Winkle, Stede, and Kreutel [WSK26] looked at generative information extraction from biographical sources using open-source large language models and found that while LLM-based extraction from biographical text is feasible, recall, false positives,

and evaluation design all present real challenges. This finding is consistent with what this thesis found. Education and clearly stated author-level fields achieved strong results, while position-related fields, institution names, department boundaries, and locations remained difficult. The two studies are not directly comparable. They use different data, different schemas, and different evaluation setups, but they point in the same direction: extraction from biographical text with large language models works better when the output format is constrained, missing values are handled explicitly, and the errors are examined carefully rather than summarized in aggregate metrics alone.

Overall, the findings show that academic biographies contain valuable researcher-focused information that structured bibliographic metadata often misses. The data that is extracted is not yet a complete knowledge graph, but it provides a structured foundation. With more validation, normalization, and linking, it could make a useful contribution to future knowledge graph enrichment.

7.2 Limitations

Several limitations are worth keeping in mind when reading the results. The first is that extraction quality is bounded by what the biography actually contains. If a biography does not mention a degree, a position, a year, or an institution, the pipeline has no way to recover that information without reaching outside the text. A missing value in the output may therefore say more about what the biography left out than about anything the model got wrong.

The second limitation is that the two data sources are quite different from each other. IEEE biographies tend to follow a more consistent style, while ORCID biographies are written and maintained by individual researchers and vary much more in structure, length, and detail. This difference in source quality feeds directly into the evaluation results and is the main reason ORCID performance generally comes out lower than IEEE performance.

A third limitation relates to how IEEE biographies were preprocessed. For each IEEE entry, we chose the longest available biography for each `dblp_pid` as the representative one. This approach was practical for large-scale processing since longer biographies usually include more career details. Still, a longer biography is not always more complete, correct, or up to date. Sometimes, a shorter biography might have newer or more accurate information.

The fourth limitation concerns the size of the gold standard. The evaluation is based on 150 manually annotated biographies, which is enough to support field-level analysis but is still a relatively small slice of the full dataset. A larger gold standard would make it possible to draw stronger conclusions about how the pipeline behaves across different disciplines, biography styles, countries of origin, and less common career patterns.

The fifth limitation is language coverage. The evaluation focuses on English biography texts, and non-English biographies fall outside the main scope of the thesis. This matters particularly for ORCID, where researcher-maintained profiles sometimes contain text in multiple languages.

The sixth limitation is that the thesis stops short of full knowledge graph integration. The extracted output is structured and compatible with graph-based formats, but turning it into something that could be integrated into a production-level scholarly knowledge graph would require additional work, such as ontology mapping, entity linking, institution name normalization, duplicate resolution, and checks for temporal consistency. Those steps are left for future work.

7.3 Future Work

There are several directions in which the pipeline could be improved in future work.

One of the most important is stronger hallucination control. At the moment, the pipeline has no way to verify that an extracted value actually appears in the source biography. Adding a step that checks extracted values against the original text, accepting them only when they are supported by exact or near-exact evidence, would make the output more trustworthy. This would be particularly valuable for location fields, where the model sometimes draws on institution names to infer a location that the biography never explicitly states.

A second direction is better entity normalization and linking. Institution names, department names, degree titles, roles, and locations currently appear in whatever form the biography uses, which means spelling variation, abbreviations, and alternative names for the same entity can all end up in the output. Linking these values to external authority sources or established knowledge bases would reduce that kind of inconsistency and make the extracted data easier to work with downstream.

A third area worth addressing is the handling of parenthetical information within named entities. During error inspection, some institution names were found to have lost content that appeared in parentheses in the original text, such as acronyms, organizational units, campus labels, or alternative names. This could be fixed through more explicit prompt instructions telling the model to preserve parenthetical text when it is part of a named entity, or through post-processing steps that normalize institution names more carefully.

Expanding the gold standard would also strengthen future evaluations. The current set of 150 annotated biographies supports field-level analysis but leaves some parts of the schema, awards, memberships, projects, activities, with too few examples to evaluate confidently. Adding more annotated biographies covering a wider range of disciplines, biography styles, languages, and career patterns would allow the pipeline to be assessed more thoroughly across different researcher groups.

Looking further ahead, the pipeline could be extended toward full knowledge graph construction. The extracted author, education, and position records are already structured in a way that lends itself to conversion into RDF triples. With suitable ontology alignment and linking to existing dblp metadata, it would be possible to combine the publication-centered information that dblp already holds with the researcher-centered career details extracted from biographies.

Finally, before any real knowledge graph integration takes place, a human-in-the-loop validation step would be worth building in. The extracted information describes researcher identities and career histories, which means errors have real consequences. Uncertain or low-confidence extractions should go through a review process rather than being added to a scholarly knowledge graph without any human check.

7.4 Conclusion

This thesis set out to investigate whether large language models could be used to extract structured researcher career information from academic biography text. The pipeline that was developed brings together preprocessing, schema-driven prompting, large-scale LLM inference, output validation, and field-level evaluation into a single end-to-end workflow.

The results show that the approach holds up well for many types of structured information, particularly fields that are clearly defined in the biography and education records that tend to follow recognizable patterns. At the same time, the work makes clear that position extraction, open-ended fields, institution name normalization, and location handling are genuinely difficult problems that are not fully resolved by prompt design alone.

The main contribution of the thesis is a scalable, schema-driven extraction workflow that takes heterogeneous academic biography text and turns it into structured author, education, and position records. The extracted data is not plugged directly into a knowledge graph here, but it provides a solid and practical foundation for future work that aims to enrich hybrid knowledge graphs by combining structured bibliographic metadata with the career details that biography text contains.

A. Extraction Prompt

This appendix contains the complete system prompt used for LLM-based career information extraction from IEEE and ORCID researcher biographies. The prompt was passed as the system message to the *SauerkrautLM-70B* model at every inference call, with only the biography text varying per request in the user message. The final version shown here is `prompt.txt`, which resulted from the iterative prompt refinement described in Section 4.3.

The prompt is structured in three parts: (1) a task definition and output constraint block specifying the JSON-only output requirement and the null-handling rule; (2) strict extraction rules covering the main entity types, edge cases, and disambiguation scenarios; and (3) the target JSON schema defining all required output fields and their types. A high-level description of how biography content maps to the schema fields is given in Section 4.2. The same prompt structure was used for both IEEE and ORCID biographies, with only the source description adapted where necessary.

System Prompt

```
You are an expert academic assistant extracting researcher
   career
information from IEEE and ORCID author biographies for
   knowledge
graph construction.

INPUTS:
- biography: full text of one author biography

OUTPUT: ONLY a valid JSON object. No markdown. No
   explanation.
No text before or after.

CRITICAL: You MUST output EVERY single key listed in the
   SCHEMA
exactly as written. If you have no data for a field, set its
    value
to null (or an empty list []), but NEVER delete or omit the
   key.
You MUST include "is_current" for every position and
   education
entry. You MUST include the "scholarly_metrics" list.
```

```
==================================================
STRICT EXTRACTION RULES
==================================================

RULE 1 - NEVER HALLUCINATE
Only extract roles, institutions, and facts explicitly
   stated or
clearly implied by the biography. If unsure, use null. Never
invent institution names, roles, or details not present in
   the
biography text.

RULE 2 - RESOLVE IMPLICIT INSTITUTION REFERENCES
  - "before joining X" / "prior to X"
    -> create a past position, is_current: false
    -> if no title is stated, use role: null and capture the
       phrase in description
  - "postdoc" / "postdoctoral"
    -> use the exact title from the biography as the role
  - "consultant" / "consulting"
    -> role: "Consultant"
  - "startup" / "founded a company"
    -> role: "Co-Founder"
  - If the institution is implied but not clearly named,
    set institution: null

RULE 3 - POSITIONS
- Include ALL roles: past and present, academic and non-
   academic
- Each role is a SEPARATE position entry
- If no role title is explicitly stated -> role: null
- Use description to capture position-related information
   that
  does not fit cleanly into role/institution
- "before joining X" -> SEPARATE past position, is_current:
   false
- Do NOT use affiliations or any external source -- extract
  institution, department, and location ONLY from biography
     text

- institution: Extract ONLY the main degree-granting or
   employing
  university/company name.
  If a research centre, lab, or department appears before
     the
  institution (e.g., "National Engineering Research Center
     of
  Electromagnetic Radiation Control Materials, UESTC"):
    -> institution: "UESTC"
    -> department: "National Engineering Research Center of
                    Electromagnetic Radiation Control
                       Materials"
```

```
- department: Extract the specific department, faculty,
   school,
  lab, or research centre within the institution.
  If a sub-unit is named before the main institution, place
     it
  here.

- location: Extract ONLY if a city or country is explicitly
  written in the biography text for that specific position
     entry.

  STRICT: location MUST be null unless the biography text
  contains an explicit geographic word directly associated
     with
  this specific role.

  Do NOT infer or derive location from:
    - The institution name itself
    - General world knowledge about where the institution is
    - Any other position or degree entry in the same
       biography
    - The author's nationality or birth location

  WRONG:
    Bio: "She was a Senior Fellow at MIT."
    -> location: "Cambridge, MA"   WRONG

  CORRECT:
    Bio: "She was a Senior Fellow at MIT, Cambridge, MA."
    -> location: "Cambridge, MA"   CORRECT

  If no location is explicitly stated -> location: null

ROLES THAT MUST ALWAYS GO INTO POSITIONS:
  - "Principal Investigator" / "Co-Investigator" / "PI"
  - "Postdoctoral Fellow" / "Postdoctoral Researcher" /
    "Research Fellow"
  - "Visiting Scholar" / "Visiting Researcher" /
    "Visiting Professor" / "Senior Visiting Scholar"
  - "Director of X" / "Head of X" / "Coordinator of X"
  - "Founder" / "Co-Founder"
  - "Editor" / "Associate Editor" / "Guest Editor" of a
     journal
  - "Advisor" / "Advisory Board Member" for a company,
    startup, or government body

Do NOT place in positions:
  - "Ph.D. Student", "Ph.D. Candidate", or "Graduate Student
     "
    -> merged into education.
  - Session chairing and one-time events -> activities.
```

```
VISITING AND FELLOWSHIP ROLES:
  - Use exact title from biography as role
  - is_current: false if dates given or clearly past tense
  - is_current: true only if biography clearly indicates
     current

RULE 4 - is_current MUST BE true OR false, NEVER null
  - true  -> "since YYYY" / "currently" / "is a" / "serves
     as"
  - false -> end year given / "was", "formerly", "prior to",
             "before joining"
  - DEFAULT: present tense with no end date -> true

RULE 5 - EDUCATION
- Copy degree wording EXACTLY from the biography.
- SPLIT COMBINED TEXT: "Bachelor in Electrical Engineering"
    -> degree: "Bachelor"
    -> field:  "Electrical Engineering"
- Use null if degree type is not clearly stated.
- Standardize university names when possible.
- Only formal degree programs go into education.
- is_current: MUST be true OR false
  - true  -> "currently pursuing", "working toward",
             "Ph.D. candidate", "Ph.D. student"
  - false -> "received", "earned", "awarded", graduation
     year
- MERGE RULE FOR STUDENTS: "Ph.D. Student", "Ph.D. Candidate
   ",
  or "Master's Student" goes ONLY in education. Do NOT
     create
  a duplicate entry in positions.
- Scholarships -> awards, NOT education.
- Do NOT put postdoctoral, visiting, or fellowship roles
  into education.
- university: Extract ONLY the main degree-granting
   university.
  If a sub-institute appears before the university name:
    -> university: main university ONLY
- field: Extract academic subject/discipline. null if none.
- location: Extract ONLY if explicitly written in biography.
  Do NOT infer from university name or world knowledge.


RULE 6 - ACTIVITIES
Activities are one-time contributions, service roles, or
   funded
participations:
  - Workshop attendance
  - Session chairing
  - Reviewer for a journal or conference
  - Teaching not tied to a formal role
```

```
  - Hosting or leading a funded programme or grant ->
     activities
  - Keynote / invited talks -> raw_facts, NOT activities

Do NOT place formal editor roles, formal positions, or long-
   term
memberships in activities.

RULE 7 - PROJECTS
- List each named research project the author worked on.
- Do NOT include funding body names alone unless a specific
  project title is given. These go to activities instead.
- Do NOT include theses here -> raw_facts.
- Generic wording ("many funded projects") -> raw_facts.

RULE 8 - MEMBERSHIPS
Formal affiliations with professional bodies and committees:
  - IEEE, ACM, SIAM, IAPR, etc.
  - Include grade when stated: "Senior Member, IEEE"
  - Editorial board membership (non-named-editor roles)
  - Advisory roles for academic societies or institutions
  - Conference organizing committees

Do NOT place named editor roles here -> these go to
   positions.

RULE 9 - PERSONAL DETAILS
Scan beginning of biography for birth information.
- Birth year     -> personal.birth_year (integer only)
- Birth location -> personal.birth_location
- nationality    -> personal.nationality
  - Extract directly if explicitly stated.
  - MAY infer from birth_location ONLY IF biography says
    "born in [country/city]".
    Example: "born in Liaoning, China" -> "Chinese"
  - Do NOT infer from university or work location.
- Do NOT put birth details in raw_facts.

RULE 10 - SCHOLARLY METRICS
Extract quantitative scholarly output metrics.
- metric_type: "publications", "books", "patents", "
   citations",
               "h_index", or "other"
- count: integer only. Convert number words ("ten" -> 10).
         Use null if no exact number given.
- description: exact short phrase giving context.
- SUM counts for the SAME category into one entry.

Categories:
1. publications: papers, articles, journal papers,
   proceedings
2. books: books, textbooks, book chapters, monographs
```

```
3. patents: patents, inventions, US patents
4. citations: citations, times cited
5. h_index: h-index, h index (count is the index number)
6. other: any other quantitative metric not fitting above

RULE 11 - AWARDS
- name: exact name of award or honor
- year: integer, or null if not mentioned
- Do NOT include academic degrees -> education
- Do NOT include research grants or funding -> projects/
   activities
- Only include prestigious PERSONAL grants/fellowships
   explicitly
  described as competitive awards.
- Do NOT repeat in raw_facts once captured here.

TIEBREAKER:
  - Named prize or personal honour       -> awards
  - Named research project (researcher)  -> projects
  - Funded programme (hosted/led)        -> activities
  - Unclear or generic                   -> raw_facts
  Never place same item in more than one field.

RULE 12 - RESEARCH AREAS
- Extract specific academic fields, topics, and domains.
- Extract ONLY the topic itself, not wrapper phrases like
  "research interests include" or "works in the area of".
- Each distinct area is a separate string in the list.

RULE 13 - RAW FACTS (safety net)
Do NOT put publication counts, citation metrics, h-index, or
book authorship here -> RULE 10.
Do NOT repeat anything already in research_areas, awards,
projects, memberships, or scholarly_metrics.
Do NOT put research topic statements here -> research_areas.
Use raw_facts ONLY for leftover facts that fit no other
   field:
  - Thesis titles or topics
  - Generic funded work statements without project names
  - Student supervision counts
  - Lab founding
  - Keynote or invited talks
  - Spin-off companies
  - Years of experience with no domain
  - Contact information

==================================================
SCHEMA
==================================================
{
  "full_name": "string",
  "personal": {
```

```
    "birth_year": "integer|null",
    "birth_location": "string|null",
    "nationality": "string|null"
  },
  "education": [
    {
      "degree": "string|null",
      "field": "string|null",
      "university": "string|null",
      "location": "string|null",
      "edu_year": "integer|null",
      "is_current": "boolean"
    }
  ],
  "positions": [
    {
      "role": "string|null",
      "department": "string|null",
      "institution": "string|null",
      "location": "string|null",
      "start_year": "integer|null",
      "end_year": "integer|null",
      "is_current": "boolean",
      "description": "string|null"
    }
  ],
  "scholarly_metrics": [
    {
      "metric_type":
        "publications|books|patents|citations|h_index|other
           ",
      "count": "integer|null",
      "description": "string"
    }
  ],
  "research_areas": ["string"],
  "awards": [
    {
      "name": "string",
      "year": "integer|null"
    }
  ],
  "projects":     ["string"] | null,
  "memberships":  ["string"] | null,
  "activities":   ["string"] | null,
  "raw_facts":    ["string"] | null
}
```

Listing A.1: Final extraction system prompt used for IEEE and ORCID biography extraction.

B. Python Packages and Standard Modules

Table B.1 summarizes the main Python packages and standard modules used during preprocessing, extraction, validation, export, and evaluation.

Table B.1: Python packages and standard modules used in the implementation.

Package or module	**Purpose**
`vllm`, `torch`	Running high-throughput LLM inference with the selected 70B model in the HPC environment.
`huggingface_hub`, `accelerate`	Downloading model files and setting up the model/inference environment.
`pandas`, `numpy`	Loading, transforming, merging, and exporting structured data during preprocessing, CSV export, and evaluation.
`scikit-learn`, `difflib`	Computing evaluation metrics and performing selected string-similarity comparisons.
`openpyxl`	Reading and writing Excel review files used during evaluation and manual inspection.
`ftfy`, `re`	Cleaning biography text, fixing encoding issues, and applying regular-expression-based normalization.
`json`, `sqlite3`	Reading and writing JSON/JSONL files and accessing the SQLite preprocessing database.
`pathlib`, `argparse`, `time`, `datetime`	File-path handling, command-line argument parsing, runtime logging, and timestamp generation.

References

[ABB+24] Marcel R. Ackermann, Hannah Bast, Benedikt Maria Beckermann, Johannes Kalmbach, Patrick Neises, and Stefan Ollinger. The dblp knowledge graph and SPARQL endpoint. *Transactions on Graph Data and Knowledge (TGDK)*, 2(2):3:1–3:23, 2024. doi:10.4230/TGDK.2.2.3.

[BILG+22] Alessandro Barducci, Simone Iannaccone, Valerio La Gatta, Vincenzo Moscato, Giancarlo Sperlì, and Sergio Zavota. An end-to-end framework for information extraction from italian resumes. *Expert Systems with Applications*, 210:118487, 2022. doi:10.1016/j.eswa.2022.118487.

[dbl25] dblp Team. dblp computer science bibliography: Monthly snapshot xml release of december 2025, 2025. doi:10.4230/DBLP.XML.2025-12-01.

[DDL+24] John Dagdelen, Alexander Dunn, Sanghoon Lee, Nicholas Walker, Andrew S. Rosen, Gerbrand Ceder, Kristin A. Persson, and Anubhav Jain. Structured information extraction from scientific text with large language models. *Nature Communications*, 15(1):1418, 2024. doi:10.1038/s41467-024-45563-x.

[GM23] Chengguang Gan and Tatsunori Mori. A few-shot approach to resume information extraction via prompts. In Elisabeth Métais, Farid Meziane, Vijayan Sugumaran, Warren Manning, and Stephan Reiff-Marganiec, editors, *Natural Language Processing and Information Systems*, pages 445–455, Cham, 2023. Springer Nature Switzerland. doi:10.1007/978-3-031-35320-8_32.

[GWD26] GWDG. KISSKI Application Process: Documentation for HPC. https://docs.hpc.gwdg.de/start_here/kisski_application_process/index.html, 2026. Accessed: 2026-06-10.

[KLZ+23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, Koblenz, Germany, 2023. ACM. doi:10.1145/3600006.3613165.

[LSZL21] XiaoWei Li, Hui Shu, Yi Zhai, and ZhiQiang Lin. A method for resume information extraction using BERT-BiLSTM-CRF. In *2021 IEEE 21st International Conference on Communication Technology (ICCT)*, pages 1437–1442. IEEE, 2021. doi:10.1109/ICCT52962.2021.9657937.

[Met24] Meta AI. Meta LLaMA 3.1 70B Instruct Model Card. https://huggingface.co/meta-llama/Llama-3.1-70B-Instruct, 2024. Accessed: 2026-06-03.

[ORC25] ORCID. ORCID Public Data File 2025, 2025. doi:10.23640/07243.30375589.

[PTG25] Fina Polat, Ilaria Tiddi, and Paul Groth. Testing prompt engineering methods for knowledge extraction from text. *Semantic Web*, 16(2):SW–243719, 2025. doi:10.3233/SW-243719.

[STA+24] Weihang Su, Yichen Tang, Qingyao Ai, Changyue Wang, Zhijing Wu, and Yiqun Liu. Mitigating entity-level hallucination in large language models. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, pages 23–31, Tokyo, Japan, 2024. ACM. doi:10.1145/3673791.3698403.

[VAG24] VAGO Solutions. Llama-3.1-SauerkrautLM-70b-Instruct. https://huggingface.co/VAGOsolutions/Llama-3.1-SauerkrautLM-70b-Instruct, 2024. Accessed: 2026-05-31.

[WSK26] Robin Winkle, Manfred Stede, and Jörn Kreutel. Generative information extraction from biographical sources. In *Proceedings of the 10th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature*, pages 311–322, Rabat, Morocco, March 2026. Association for Computational Linguistics. doi:10.18653/v1/2026.latechclfl-1.30.

[XCP+24] Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang, and Enhong Chen. Large language models for generative information extraction: A survey. *Frontiers of Computer Science*, 18(6):186357, 2024. doi:10.1007/s11704-024-40555-y.

[ZWC+24] Yuqi Zhu, Xiaohan Wang, Jing Chen, Shuofei Qiao, Yixin Ou, Yunzhi Yao, Shumin Deng, Huajun Chen, and Ningyu Zhang. LLMs for knowledge graph construction and reasoning: Recent capabilities and future opportunities. arXiv preprint arXiv:2305.13168, 2024. doi:10.48550/arXiv.2305.13168.

Affidavit

I hereby declare that I have written this master's thesis independently and have used no sources or resources other than those cited, and that I have clearly identified any ideas taken directly or indirectly from external sources as such. I have not previously submitted this thesis to any other examination office in the same or a comparable form. It has also not been published to date. Trier, 17 June 2026